

CSE 341 — Sebesta Bölüm Sonu Soruları: Ch 8, 15, 16

Çözümlü çalışma notu. Sebesta *Concepts of Programming Languages*, 12. baskıdan, finalin kapsamındaki üç bölümün Review Questions + Problem Set + Programming Exercises bölümleri.

Kapsam:

- **Ch 8** Statement-Level Control Structures (52 soru)
- **Ch 15** Functional Programming Languages (82 soru — Scheme'e odaklı; ML/Haskell/F# soruları kısa)
- **Ch 16** Logic Programming Languages (42 soru)

Stil: Her soru için (1) İngilizce orijinal, (2) Türkçe çeviri/yorum, (3) çözüm. Kod örnekleri orijinal sözdiziminde (Scheme, Prolog, C).

Chapter 8 — Statement-Level Control Structures

Review Questions (31 soru)

1. What is the definition of control structure?

TR: *Control structure tanımı nedir?*

Bir **control statement** (kontrol deyimi), kendi başına bir hesaplama yapmak yerine, programın çalışma akışını yönlendirir — yani başka deyimlerin ne zaman ve hangi sırayla yürütüleceğini belirler. Bir **control structure** ise bir control statement artı bu deyim yönettiği deyim/deyim bloğunun toplamıdır. Örnek: `if (cond) body` ifadesi tek başına bir control structure'dır; `if` control statement'tır, `body` ise yönetilen kısımdır.

2. What did Böhm and Jacopini prove about flowcharts?

TR: *Böhm ve Jacopini, akış diyagramları (flowchart'lar) hakkında ne kanıtladı?*

1966'daki teoremlerinde, herhangi bir akış diyagramının **yalnızca üç kontrol yapısı** kullanılarak yeniden ifade edilebileceğini gösterdiler:

1. **Sequence** (sıra) — deyim ardı ardıncılığı
2. **Selection** (seçim) — if/else
3. **Iteration** (yineleme) — while döngüsü

Bu sonuç **structured programming** akımının teorik temelidir: goto olmadan da her algoritma yazılabilir.

3. What is the definition of block?

TR: *Block (blok) tanımı nedir?*

Bir **block**, içinde değişken bildirimleri de bulundurabilen, mantıksal olarak tek bir compound statement (bileşik deyim) gibi davranan kod parçasıdır. ALGOL 60'tan beri var. C ailesinde { ... } ile, Pascal/Ada'da begin ... end ile sınırlanır. Bir bloğun temel özelliği kendi **scope**'unu (kapsamını) yaratabilmesidir.

4. What is/are the design issue(s) for all selection and iteration control statements?

TR: *Tüm seçim ve yineleme kontrol deyimleri için tasarım sorunları nelerdir?*

Sebesta'nın listesi, hem selection hem iteration için ortak geçerli olan temel sorulardır:

1. **Control expression'ın tipi ne olmalı?** Boolean mı, aritmetik mi, başka mı?
2. **Kontrol edilen statement tek bir statement mı yoksa bir statement segment (blok) mı olabilir?**
3. **Nesting (iç içe yerleştirme) nasıl ele alınır?** Dangling else, label sıralaması gibi sorunlar.

5. What are the design issues for selection structures?

TR: *Seçim yapıları için tasarım sorunları nelerdir?*

Two-way (if-else) ve multiple selection (switch/case) için ayrı ayrı sorulur:

Two-way selection (if):

1. Then ve else kısmı tek statement mı yoksa blok mı? (C: tek; çoklu için { } gerekli)
2. Selector expression Boolean mı, arithmetic mi? (Java: yalnız Boolean; C: arithmetic da olur)

3. Nesting'de else hangi if'e bağlanır? (**Dangling else** problemi)

Multiple selection (switch):

1. Selector expression'ın izinli tipleri? (C: integral; Java 7+: String da; Ada: discrete tipler)
2. Case değerleri sabit mi olmak zorunda?
3. Bir kod parçası birden fazla değer için seçilebilir mi? (C: fall-through, Ada: liste)
4. Bir case sonraki case'e implicit olarak akar mı? (C: evet; Ada: hayır)
5. Default (catch-all) clause var mı?

6. What is unusual about Python's design of compound statements?

TR: *Python'un bileşik deyim tasarımının olağandışı yönü nedir?*

Python, blokları **küme parantezi veya keyword** ile değil, **girintilemeyle (indentation)** belirtir. Aynı seviyedeki kod aynı bloğa aittir; girinti azalınca blok biter.

```
if x > 0:
    print("pos")
    y = x * 2
print("done")    # bu artık if dışında
```

Bu, programcıyı temiz girintilemeye zorlar (readability artar), ama copy-paste'te tab/space karışıklığı bug kaynağı olabilir.

7. Under what circumstances must an F# selector have an else clause?

TR: *F#'ta selector ne zaman else clause'una sahip olmak zorundadır?*

F#'ta `if`, **bir expression**'dır (statement değil) — yani bir değer döner. Dolayısıyla `if` bir değişkene atanıyorsa veya bir ifade içinde kullanılıyorsa, hem `then` hem `else` kollarının bir değer döndürmesi şarttır:

```
let y = if x > 0 then 1 else -1    // OK
let y = if x > 0 then 1           // HATA – else olmadan değer döndüremez
```

Yalnızca `unit` (void benzeri) bir değer dönecekse, else opsiyonel olabilir.

8. What are the common solutions to the nesting problem for two-way selectors?

TR: İki yollu seçim deyimlerinde nesting sorunu için yaygın çözümler nelerdir?

Dangling else problemi: `if (a) if (b) S1 else S2 — else hangi if 'e bağlı?`

Yaygın çözümler:

1. **En yakın eşleşmeyen if'e bağla** (C, C++, Java default). Pratik ama programcı girintilemeden yanılabilir.
2. **Mecburi blok** — her `if - then` ve `else` blok içinde olsun (modern stil önerisi).
3. **Explicit kapanış keyword'ü** — Ada `end if`, Pascal `begin/end`. Dangling else kavramı doğmaz.
4. **F#/ML stili** — `if cond then expr1 else expr2` tek satırda; ambiguity yok.

9. What are the design issues for multiple-selection statements?

TR: Çoklu seçim deyimleri için tasarım sorunları nelerdir?

Yukarıda Soru 5'in switch kısmında listelediğimle aynı:

1. Selector'ın tipi ve değer türleri.
2. Case-değerleri (constants), tekil mi, range mı, liste mi?
3. Bir case'e birden fazla değer atanabilir mi?
4. Implicit fall-through var mı?
5. Default case zorunlu mu, isteğe bağlı mı, ortada mı sonda mı?
6. Selector değeri hiçbir case'e uymazsa ne olur?

10. Between what two language characteristics is a trade-off made when deciding whether more than one selectable segment is executed in one execution of a multiple selection statement?

TR: Bir multiple selection deyiminin bir çalıştırılışında birden fazla seçilebilir bloğun yürütülüp yürütülmeyeceğine karar verirken hangi iki dil özelliği arasında bir trade-off yapılır?

Writability vs Reliability:

- **Writability:** Fall-through (C tarzı) yazımı kolaylaştırır. Birden fazla case'in aynı kodu paylaşması için sadece case label'ları üst üste konur.
- **Reliability:** Fall-through olmazsa (Ada tarzı), unutulmuş `break` kaynaklı sessiz bug'lar olmaz; kod sezgisel davranır.

C, writability'yi seçti; Ada (ve Java 14+ switch expression'ları) reliability'yi.

11. What is unusual about C's multiple-selection statement?

TR: *C'nin çoklu seçim deyiminin olağandışı yönü nedir?*

Implicit fall-through: Her `case` 'de `break` yoksa, bir sonraki case'in kodu da çalışır. Programcı yanlışlıkla `break` koymayı unutursa sessiz bug doğar.

```
switch (x) {  
    case 1: do_a();    // break YOK → case 2'ye düşer  
    case 2: do_b(); break;  
}  
// x == 1 ise hem do_a hem do_b çalışır
```

İkinci olağandışılık: `case` etiketleri herhangi bir noktaya konabilir — bazı esoteric kullanımlara izin verir (Duff's device).

12. On what previous language was C's switch statement based?

TR: *C'nin switch deyimi hangi daha önceki dile dayanır?*

ALGOL 68'in `case / out` yapısına. (Dolaylı olarak Fortran computed goto'ya da benzer.) Pratik tasarımı doğrudan **BCPL** ve **B** (C'nin doğrudan ataları) üzerinden geldi.

13. Explain how C#'s switch statement is safer than that of C.

TR: *C#'ın switch deyimi C'ninkinden niye daha güvenli, açıklayın.*

C#'ta **implicit fall-through YOKTUR**. Her non-empty case açıkça nasıl biteceğini söylemelidir:

- `break;` — switch'ten çık
- `goto case <label>;` — başka bir case'e atla (explicit)

- return , throw , continue — diğer çıkış yolları

Sonraki case'e düşmek için goto case -i açıkça yazmak gerekir. Bu sayede:

- "Break yazmayı unuttum" tipi sessiz bug imkânsız hâle gelir.
- Birden fazla değer için aynı kodu paylaşmak boş case'ler ile mümkün (Java/C gibi):

```
case 1: case 2: do_x(); break;
```

14. What are the design issues for all iterative control statements?

TR: *Tüm yinelemeli kontrol deyimleri için tasarım sorunları nelerdir?*

1. **Yinelemenin nasıl kontrol edileceği:** counter (counter-controlled), logical condition (logically controlled), ya da kombinasyon.
2. **Loop variable nereden kontrol ediliyor:** dilde mi (Ada) yoksa programcı mı (C)?
3. **Pre-test mi post-test mi?** (Şartı başta mı sonda mı kontrol et.)
4. **Loop'un yapısı içinde nasıl çıkış imkânları var?** break, return, exit when ...
5. **Loop body içinde loop variable değişebilir mi?**
6. **Çıkıştan sonra loop variable ne değerde?**

15. What are the design issues for counter-controlled loop statements?

TR: *Counter-controlled loop deyimleri için tasarım sorunları nelerdir?*

Counter-controlled loop (ör. for i := 1 to 10) için özgün sorular:

1. **Loop variable'in tipi?** (integer, real, enumeration, ...)
2. **Loop parametreleri (başlangıç, son, adım) ne zaman hesaplanır?** Bir kez mi, her iterasyonda mı? (Ada: bir kez; C: her iterasyonda condition expression değerlendirilir.)
3. **Loop variable scope'u nedir?** Loop dışında erişilebilir mi? (C99+: bloğa ait; eski C: erişilebilir.)
4. **Loop body içinde loop variable veya range parametreleri değiştirilebilir mi?** (Ada hayır; C evet.)
5. **Loop bitince loop variable ne olur?** (C: condition'ı false yapan son değer; Pascal: undefined.)

16. What is a pretest loop statement? What is a posttest loop statement?

TR: *Pretest ve posttest loop deyimleri nedir?*

- **Pretest:** Şart, her iterasyondan **önce** kontrol edilir. Şart false ise body **hiç çalışmayabilir**. Örnek:
`while (cond) body .`
- **Posttest:** Şart, her iterasyondan **sonra** kontrol edilir. Body **en az bir kez** çalışır. Örnek:
`do body while (cond); (C), repeat body until cond (Pascal).`

Karar kriteri: Body en az bir kez çalışmalı mı? Evetse posttest mantıklı; değilse pretest.

17. What is the difference between the for statement of C++ and that of Java?

TR: *C++ ve Java'nın for deyimi arasındaki fark nedir?*

İkisi de aynı şablonu kullanır — `for (init; cond; update) body` — ama:

- **C++:** Init kısmında tek bir tip tanımlanabilir, fakat aynı satırda farklı tipte değişken zor (workaround mevcut). Loop variable scope C++'ta loop'a yereldir (modern C++).
- **Java:** Loop variable her zaman blok-scope'lu (loop sonunda erişilemez). Java ek olarak `for-each` formunu da destekler:

```
for (int x : array) { ... }
```

Pratik fark: Java'nın **enhanced for** (foreach) syntax'i Java 5'te dile entegre olmuştur; C++'ta range-based for C++11'de geldi.

18. In what way is C's for statement more flexible than that of many other languages?

TR: *C'nin for deyimi diğer pek çok dile göre hangi açılardan daha esnektir?*

C'nin `for (init; cond; update) body` formu, "counter-controlled" şablonun çok ötesine geçer:

1. **Üç parça da boş bırakılabilir:** `for (;;) {}` → sonsuz döngü.
2. **Virgül operatörü ile çoklu init/update:** `for (i=0, j=n; i<j; i++, j--)` .

3. **Cond herhangi bir ifade:** Sadece counter test değil, herhangi bir Boolean. Örneğin:

```
for (p = head; p != NULL; p = p->next) .
```

4. **Update keyfi bir ifade:** Sayı arttırmak zorunda değil.

5. **Body içinde counter değiştirilebilir.**

Tüm bu esneklik C'nin yüksek **writability**'sinin kaynağı ama aynı zamanda **reliability** kaybı.

19. What does the range function in Python do?

TR: *Python'da range fonksiyonu ne yapar?*

`range(start, stop, step)` , ardışık tam sayılardan oluşan bir **lazy sequence** üretir — `stop` dahil DEĞİLDİR.

```
range(5)           # 0, 1, 2, 3, 4
range(1, 6)        # 1, 2, 3, 4, 5
range(0, 10, 2)    # 0, 2, 4, 6, 8
range(10, 0, -1)   # 10, 9, 8, ..., 1
```

Python 3'te `range` bir generator (üreteç) gibi davranır — değerleri bellek tutmadan tek tek üretir. Bu yüzden `for i in range(10000000):` bellekte 10M sayı tutmaz.

20. What contemporary languages do not include a goto?

TR: *Hangi modern diller goto içermez?*

- **Java** — `goto` reserved keyword'tür ama dilde kullanılmıyor (saklı tutulmuş).
- **Python** — yoktur.
- **Ruby** — yoktur (3. parti kütüphane ile workaround vardır ama dilin parçası değil).
- **Scheme, Haskell, ML, F#** — functional dillerde yok (zaten gerekmez).
- **JavaScript** — yoktur (ama `break/continue` label'lı vardır).

Goto'su olanlar: C, C++, C#, Go, Perl, BASIC, Fortran, Pascal, Ada.

21. What are the design issues for logically controlled loop statements?

TR: *Mantıksal olarak kontrol edilen döngü deyimleri için tasarım sorunları nelerdir?*

1. **Şart, body'den önce mi sonra mı kontrol edilir?** (pre vs post)
2. **Yapı doğrudan ayrı bir deyim mi, yoksa counter loop'un özel hâli mi?** (Bazı diller `while` 'ı yalnız sözel olarak `for` 'tan ayırır.)
3. **Loop body'sinden ortadan çıkış mekanizması var mı?** (`break`, `Ada exit when`, ...)

22. What is the main reason user-located loop control statements were invented?

TR: *Kullanıcı yerleştirmeli (user-located) loop kontrol deyimleri (break/continue/exit) niye icat edildi?*

Loop'un ortasında çıkış ihtiyacını karşılamak. Pure structured programming'te döngü, ya başta (pre-test) ya da sonda (post-test) kontrol edilir. Ama bazı algoritmalarda — örneğin "ara, bulunca dur" — çıkış noktası loop'un ortasındadır:

```
while (1) {  
    item = read_next();  
    if (item == END) break;      // ortadan çıkış  
    process(item);  
}
```

Goto kullanmadan bu deseni temiz ifade etmek için `break/continue/exit` eklendi.

23. What are the design issues for user-located loop control mechanisms?

TR: *User-located loop control mekanizmaları için tasarım sorunları nelerdir?*

1. **Kontrol-transfer mekanizması mı (gerçek anlamda goto-benzeri) yoksa sadece koşullu çıkış mı?**
2. **Hangi loop'tan çıkar?** Sadece en içteki mi (basit `break`), yoksa label ile belirli bir loop mu?
3. **Switch içinden çıkışta da çalışır mı?** (C'de aynı `break` her ikisi için kullanılır.)
4. **Loop dışında kullanım yasak mı?** (Yasak değilse syntax error olmaz ama mantıksızdır.)

24. What advantage does Java's break statement have over C's break statement?

TR: Java'nın break'inin C'nin break'ine göre avantajı nedir?

Labeled break. Java'da break bir label'la kullanılabilir; programcı belli bir loop'tan çıkmak istediğini belirtir:

```
outer:
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (matrix[i][j] == target) {
            found = true;
            break outer;    // dış for'dan çıkar
        }
    }
}
```

C'de sadece en içteki loop'tan çıkılır → çoklu nested loop'tan çıkmak için ya flag ya goto gerekir. Java labeled break, goto ihtiyacını ortadan kaldıran temiz alternatiftir.

25. What are the differences between the break statement of C++ and that of Java?

TR: C++'in ve Java'nın break deyimi arasındaki farklar nelerdir?

Özellik	C++	Java
Sadece en içteki loop'tan/switch'ten çıkar	✓	(label'sız da aynı)
Label desteği	X	✓ (break LABEL;)
continue label desteği	X	✓ (continue LABEL;)

Java'nın label'lı versiyonu daha esnek; C++'ta aynı sonuca goto ile ulaşılır.

26. What is a user-defined iteration control?

TR: Kullanıcı tanımlı yineleme kontrolü (iterator) nedir?

Programcının kendi koleksiyon tipi için yineleme mantığını tanımlayabildiği bir mekanizma. Genel kalıp: koleksiyon, sıradaki elemanı veren bir **iterator** nesnesi sağlar; dil-seviyesi for-each bu nesneyi kullanır.

- **Java:** Iterable<T> ve Iterator<T> arayüzleri.
- **C#:** IEnumerable<T> ve yield return .
- **Python:** __iter__ ve __next__ "magic" metotları; yield ile generator.
- **Ruby:** each ve block'lar.

Avantaj: koleksiyonun iç yapısı saklanır (encapsulation), ama yineleme yine de dil-seviyesi syntax ile yapılabilir.

27. What Scheme function implements a multiple selection statement?

TR: *Scheme'de çoklu seçim deyimini hangi fonksiyon karşılar?*

COND .

```
(COND
  ((test1) result1)
  ((test2) result2)
  ...
  (ELSE default-result))
```

Test'ler sırasıyla değerlendirilir; ilki #t veren clause'un expr'i çalıştırılır. CASE da vardır (selector değerini sabitlerle eşler), ama daha sınırlı.

28. How does a functional language implement repetition?

TR: *Functional bir dil yinelemeyi nasıl gerçekler?*

Recursion (özyineleme) ile. Imperative loop'un işlevini, fonksiyonun kendini çağırması karşılar. Pure functional dillerde mutable state yoktur → loop counter de yoktur. Tipik kalıp:

```
(DEFINE (fn lst)
  (COND
    ((NULL? lst) base-case)
    (ELSE (... (CAR lst) ... (fn (CDR lst))))))
```

Tail recursion + compiler optimization sayesinde bu, performans olarak imperative loop'la aynı (sabit stack).

29. How are iterators implemented in Ruby?

TR: *Ruby'de iterator'lar nasıl gerçekleştirilir?*

Ruby'de iterator'lar **block**'lar ve `yield` ile uygulanır. Tipik kullanım: bir koleksiyon nesnesinin metodu, her eleman için `yield` çağırır; çağırılan block bu elemanı işler.

```
[1, 2, 3].each do |x|  
  puts x * 2  
end
```

`each`, `map`, `select`, `inject`, `times`, vb. yerleşik iterator'lar sınıfların metotlarıdır. Kullanıcı kendi sınıflarına `each` (veya `Enumerable` modülünü include ederek) tanımlayarak custom iterator yazabilir.

30. What language predefines iterators that can be explicitly called to iterate over its predefined data structures?

TR: *Hangi dil, kendi yerleşik veri yapıları üzerinde gezinmek için açıkça çağrılabilen önceden tanımlı iterator'lar sağlar?*

Java — `Iterator<T>` arayüzü ve `iterator()` metodu ile.

```
Iterator<String> it = list.iterator();  
while (it.hasNext()) {  
    String s = it.next();  
    ...  
}
```

Java `foreach` (`for (T x : coll)`) bu iterator'ın syntactic sugar'ıdır. C# `IEnumerator` / `foreach` aynı modeli takip eder. Python (`iter()` ve `next()`) ve Ruby de benzer mekanizmaları sağlar.

31. What common programming language borrows part of its design from Dijkstra's guarded commands?

TR: Hangi yaygın programlama dili tasarımının bir kısmını Dijkstra'nın guarded commands'inden alır?

Ada'nın `select` deyimi (concurrent programming için kullanılan). Birden fazla guard arasında **nondeterministic** seçim Dijkstra'nın `if ... fi` ve `do ... od` yapılarına benzer.

```
select
  when condition1 => accept Entry1; ...
or
  when condition2 => accept Entry2; ...
end select;
```

Birden fazla guard true ise herhangi biri seçilebilir — Dijkstra'nın nondeterminism'ine ek olarak gerçek concurrent program kontrol yapısı.

Ayrıca **Promela** ve **CHANNELS** gibi modelleme/formal verification dilleri de guarded commands'ı doğrudan kullanır.

Problem Set (12 soru)

1. Describe three situations where a combined counting and logical looping statement is needed.

TR: Counting + logical condition bir araya getirilmiş loop'un gerektiği üç durum açıkla.

Sayma + mantıksal koşul karışımı genelde "en fazla N kere dene ama bu arada koşul gerçekleşirse dur" senaryosudur.

1. **Retry with timeout / max-attempts:** Ağ isteğini en fazla 5 kere yeniden dene, ama başarılı olursa erken çık.

```
for (int i = 0; i < 5 && !success; i++) {
    success = try_request();
}
```

2. **Arama (sınırlı boyutta veri yapısı):** Dizide hedefi ara — hem `i < n` (sınır) hem `arr[i] != target` (mantıksal şart) sağlanırken devam et.

```
for (int i = 0; i < n && arr[i] != target; i++) { }
```

3. **Numerical convergence (yakınsama):** N iterasyona kadar Newton iterasyonu yap, ama tolerance altına düşersen dur.

```
for (int i = 0; i < MAX_ITER && fabs(diff) > EPSILON; i++) {  
    x_new = update(x); diff = x_new - x; x = x_new;  
}
```

2. Study the iterator feature of CLU in Liskov et al. (1981) and determine its advantages and disadvantages.

TR: *CLU'daki iterator özelliğini Liskov'un kitabından çalış; avantaj ve dezavantajlarını belirle.*

CLU (Barbara Liskov, 1974) iterator'ları dil seviyesinde, generator'lar olarak tanıttı — sonradan gelen Python `yield`, C# `IEnumerator`, Ruby `each` 'in kavramsal atası.

CLU iterator özellikleri:

- Iterator bir prosedür gibi tanımlanır ama içinde `yield` ile çağırana değer döner.
- Bir `for x: ints = items()` kalıbı ile loop'tan kullanılır.
- Coroutine-benzeri davranış: iterator state'i saklar, sıradaki `yield` 'e kadar çalışır.

Avantajlar:

- Iteration ve traversal logic koleksiyondan ayrı; encapsulation güçlü.
- Composition kolay: bir iterator'ı filter/map iterator'larıyla kombine edebilirsin.
- Loop counter / state manuel yönetimi gereksiz.
- Lazy evaluation — sonsuz dizilerle bile çalışabilir.

Dezavantajlar:

- Implementation karmaşık (coroutine state machine gerek).
- O dönemde çok yeni bir kavramdı; debugging ve performans tooling zayıftı.
- Çoğu sistemde dile özel runtime gerektirdiğinden yaygınlaşmadı; sonradan diğer dillerde benzer formda yeniden doğdu.

3. Compare the set of Ada control statements with those of C# and decide which are better and why.

TR: Ada'nın ve C#'ın kontrol deyimlerini karşılaştır; hangisi daha iyi, neden?

Özellik	Ada	C#
if-then-else	if ... then ... elsif ... else ... end if	if ... else if ... else ... { }
Switch	case X is when V => ...; when others =>	switch (x) { case V: ...; default: }
Switch fall-through	YOK	YOK (goto case explicit)
Counter loop	for I in 1 .. N loop ... end loop	for (int i = 0; i < n; i++) { }
Loop var değiştirilebilir mi	Hayır	Evet
Logical loop	while ... loop ... end loop	while (...) { }
Posttest	loop ... exit when ...; end loop (esnek)	do { } while (...)
break	exit (label'lı: exit OUTER when ...)	break; (label yok), goto label;
goto	Var, ama disiplinli	Var
Loop variable scope	Tam loop'a yerel	Loop'a yerel (C# 1.0+)

Değerlendirme:

Reliability açısından Ada daha iyi:

- Loop variable read-only — yanlışlıkla değiştirilemez.
- exit when herhangi bir noktada koşullu çıkış sağlar → break'in genelleştirilmesi gibi.
- Explicit end if , end loop — dangling else yok, kapanış net.
- Fall-through hatası imkânsız.

Writability/familiarity açısından C# daha iyi:

- C ailesi syntax'i milyonlarca programcı için sezgisel.
- for daha esnek (init/cond/update üçü de istediğin gibi).
- goto case ile fall-through hâlâ kasıtlı yapılabilir.

Sonuç: Yüksek-güvenilirlik gerektiren sistemler (havacılık, savunma → Ada'nın doğal pazarı) için Ada üstün. Genel-amaç ve performans-odaklı işler için C#'ın esnekliği daha pratik. Modern Java, C#'ın bir kısım reliability fikirlerini (Boolean-only condition, no fall-through Java 14+) benimsedi — yani trend Ada yönünde.

4. What are the pros and cons of using unique closing reserved words on compound statements?

TR: *Bileşik deyimlerde benzersiz kapanış keyword'leri (end if, end loop, end case) kullanmanın artı ve eksileri.*

Artılar:

- Hangi yapının kapandığı tam belli. Uzun nested kodda } denizinde "bu hangisinin kapanışydı?" sorusu yaşanmaz.
- Compiler hatası daha net mesajla bulur (eksik end if → tam o noktayı işaret eder).
- Dangling else gibi syntactic belirsizlikler doğmaz.
- İnsan okurken yapıya ait kapanışı görür — implicit anlam yerine explicit.

Eksiler:

- Daha fazla yazım, daha uzun kod (verbose).
- Aynı bloğun başlangıç ve sonu farklı isimde — değiştirirken iki yeri de güncelleme gerek.
- } ile sözel kapanış arasında — modern stil rehberleri auto-indent ve match-brace araçlarının da bu işi çözdüğünü söyler.

Net değerlendirme: Eğitsel / güvenlik-kritik kodda artı; performans/hız odaklı kodda eksilerin önemi azalır.

5. What are the arguments, pros and cons, for Python's use of indentation to specify compound statements in control statements?

TR: *Python'un bileşik deyimleri girintilemeyle belirtmesinin lehte ve aleyhte argümanları.*

Lehte (pros):

- Programcıyı temiz girintilemeye zorlar → görsel yapı ve mantıksal yapı her zaman aynı.
- Brace karakterleri yok → daha az syntax noise, kod kısa ve sade.
- "İki farklı kişi aynı stille kod yazar" → standardizasyon sağlar.
- { } koymayı unutma hatası imkânsız.

Aleyhte (cons):

- Tab vs space karışımı sessiz bug'lara yol açar (Python 3 bunu hata yapsa da editörler hâlâ karıştırabiliyor).
- Copy-paste'te girintileme bozulabilir.
- Otomatik kod üreten araçlar için sıkıntılı (girintinin AST'ye gömülmesi).
- Tek satıra çoklu deyim yazmak daha zor (lambdas içinde tek expression).
- Bazı IDE'lerde girintileme yanlış görünürse mantık değişir.

Genel kabul: Python topluluğunda büyük çoğunluk "kazandı" diyor — okunabilirlik kayıtsız şartsız artı. Diğer modern diller (Nim, Haskell off-side rule) benzer fikri benimsedi.

6. Analyze the potential readability problems with using closure reserved words for control statements that are the reverse of the corresponding initial reserved words, such as the case-esac reserved words of ALGOL 68. For example, consider common typing errors such as transposing characters.

TR: *ALGOL 68'in case-esac , if-fi , do-od gibi ters-yazılmış kapanış keyword'lerinin okunabilirlik sorunlarını analiz et — örn. karakter transpoze hataları.*

ALGOL 68'in fikri: kapanış keyword'ü açılışın tersi → if/fi , case/esac , do/od .

Readability problemleri:

1. **Sözlüksel anlamsızlık:** fi , esac , od doğal dilde anlam taşımaz. end if , end case Türkçe/İngilizce okunduğunda anında anlaşılır.
2. **Yazım hataları yakalanmaz:** fi yerine if yazmak veya iki harfin yer değiştirmesi (fi → if) tipik typo'dur. Compiler tipini yanlış parse eder, hata mesajı asıl problemden uzakta çıkar.
3. **Görsel benzerlik:** fi ile if , od ile do çok yakın görünür. Code review'da gözden kaçır.
4. **Çok dilli ortam:** "esac" kullanıcının ana dilinde rastgele bir harf dizisi gibi durur; yabancı keyword olarak öğrenmek zorunlu.
5. **Tonlamasız:** "end if" vurgulu okunabilir, "fi" bir nefes alıştan ibaret.

Pratik sonuç: Sonraki diller (Ada, Bash) case/esac ve if/fi 'ı miras aldı — ama yenilerinde (Java, C#, Python, Rust) açıkça reddedildi. Modern stil "explicit kelime kapanış" (end if) ya da "blok delimiter" ({ }) yönünde gitti.

7. Use the Science Citation Index to find an article that refers to Knuth (1974). Read the article and Knuth's paper and write a paper that summarizes both sides of the goto issue.

TR: Knuth (1974) "Structured Programming with goto Statements"ı oku ve onu refere eden bir makaleyi de incele; goto tartışmasının iki tarafını özetle.

Knuth 1974 — "Structured Programming with go to Statements":

Knuth Dijkstra'nın 1968 mektubuna ("Go To Statement Considered Harmful") cevap niteliğinde yazdı. Dijkstra'nın haklı taraflarını kabul ediyor ama mutlak yasak fikrine karşı çıkıyor.

Knuth'un argümanları:

- Bazı kontrol akışı kalıpları (örn. error-handling cleanup, çoklu nested loop'tan çıkış, durum makineleri) goto olmadan **daha karmaşık** olur.
- Performans-kritik kodda (loop optimizasyonları, branch placement) goto compiler/programcıya alan açar.
- "Sıralı bir program goto kullanmadan da yazılabilir" doğru — ama "yazılmalı" değil.
- Disiplinli kullanılan goto, abstraksiyon seviyesinde kaybetmez; aksine bazı durumlarda yapıyı şeffaflaştırır.

Karşı taraf (Dijkstra ve takipçileri):

- goto kullanımı kod statici yapısı ile çalışırken kontrol akışı arasındaki **mental gap**'i büyütür → reasoning zorlaşır.
- Structured programming yapıları (sequence/selection/iteration) algoritmik düşünmeyi formal olarak destekler.
- Disiplinsiz kullanıldığında goto "spaghetti code" doğurur, bakımı imkânsızlaşır.

Modern uzlaş:

- "Yapıyı zenginleştiren" goto kalıpları (örneğin Linux kernel'in error-cleanup goto'su) **kabul edilebilir**.
- Random kontrol akışı için goto hâlâ kötüdür.
- Java/Python/Ruby goto'yu tamamen attı — daha disiplinli alternatifler (labeled break/continue, exception, return) verdi.

8. In his paper on the goto issue, Knuth (1974) suggests a loop control statement that allows multiple exits. Read the paper and write an operational semantics description of the statement.

TR: *Knuth 1974'te birden fazla çıkış noktası olan bir loop yapısı öneriyor — operational semantics tanımını yaz.*

Knuth'un önerisi: **loop ... endloop** yapısı, body'nin herhangi bir noktasında **leave** veya etiketli **leave X** ile çıkış izni verir.

Pseudosyntax:

```
loop:
  S1;
  when cond1: leave;
  S2;
  when cond2: leave;
  ...
endloop;
```

Operational semantics (informal):

Durum: (σ, c) — σ değişken atamaları, c kontrol akışı pozisyonu.

```
exec(loop body endloop,  $\sigma$ ) :
   $\sigma' = \text{exec\_body}(\text{body}, \sigma)$ 
  if exit_signal triggered during body:
    return  $\sigma'$  // loop'tan çık
  else:
    exec(loop body endloop,  $\sigma'$ ) // başa dön (recursive)
```

```
exec_body(S1; when  $\phi$  leave; S2,  $\sigma$ ) :
   $\sigma_1 = \text{exec}(S1, \sigma)$ 
  if eval( $\phi$ ,  $\sigma_1$ ) = true:
    signal exit; return  $\sigma_1$ 
  else:
    return exec_body(S2,  $\sigma_1$ )
```

Pratikte: body soldan sağa çalışır; bir **leave** görülünce signal yükselir, en yakın **loop** 'tan çıkılır. Etiketli versiyon (**leave OUTER**) en yakın etkili loop'tan çıkar.

Sonraki dillerin etkilendiği yer: **Ada** **loop ... exit when X; end loop** , **Common Lisp** **LOOP ... LOOP-FINISH** , **Perl** **last LABEL** hep bu fikrin türevleri.

9. What are the arguments both for and against the exclusive use of Boolean expressions in the control statements in Java (as opposed to also allowing arithmetic expressions, as in C++)?

TR: *Java'nın kontrol deyimlerinde sadece Boolean ifadelere izin vermesinin lehte ve aleyhte argümanları (C++'taki arithmetic'e de izin veren yaklaşıma karşılık).*

Bu Quiz 12B'nin tam cevabı — özet:

Lehte (Java'nın seçimi):

- **Readability:** `if (x)` Boolean ise niyet açık; arithmetic ise "x sıfır mı?" okunmuyor.
- **Reliability:** Klasik `if (a = b)` hatası Java'da compile error; C'de sessizce atama olur.
- **Type safety:** Tip sistemi `int` → `bool` örtük dönüşümünü reddeder → bug erken yakalanır.
- **Anlam netliği:** Boolean operator'ları (`&&`, `||`) zaten Boolean sonuç verir; system tutarlılığı.

Aleyhte (C++'ın esnekliği):

- **Writability:** `if (p)` (pointer null kontrolü) tek satır; Java'da `if (p != null)` yazmak gerek.
- **Klasik C idiomları:** `while ((c = getchar()) != EOF)` gibi atamanın değerini direkt koşula koyma — C'de standart.
- **Sayıcı kontrolü:** `if (count)` (sıfırdan büyük mü) kısa.

Sonuç: Modern dillerin çoğu (Java, C#, Python, Swift, Rust) Java tarafını seçti — reliability writability'den önemli sayılıyor. C++'ı çağırıştıran her yeni dil ayrıca compiler warning'ler ile bug kaynağı pattern'leri yakalıyor (ör. GCC `-Wparentheses`).

10. Describe a programming situation in which the else clause in Python's for statement would be convenient.

TR: *Python'un for-else yapısının işe yarayacağı bir senaryoyu açıkla.*

Python'da `for/while` 'a `else` eklenebilir: `else` body'si, loop **normal şekilde bittiğinde** (yani `break` ile değil) çalışır.

Klasik kullanım — "ara, bulamadıysan haber ver":

```
for x in collection:
    if matches(x, target):
        print("Bulundu:", x)
        break
else:
    print("Bulunamadı.")    # loop break'siz bitti
```

Geleneksel yazımda bir `found = False` flag'i taşımak gerek. `for-else` bu flag'i ortadan kaldırır.

Diğer örnekler:

- Bir asalılık testi: tüm bölenleri denedik, hiçbiri tam bölmedi → asal.
- Bir döngüde validation yap; tüm itemler valid'se "all clear" mesajı bas.

Karşı argüman: `else` keyword'ü `if/else` 'in mantığını çağırıştırıyor; `for-else`'i ilk gören biri "loop hiç çalışmamışsa mı?" sanabilir. Python topluluğunda da bu özellik biraz controversial.

11. Describe three specific programming situations that require a posttest loop.

TR: *Posttest loop'un (do-while) gerekli olduğu üç senaryo.*

Posttest loop'un belirleyici özelliği: **body en az bir kez mutlaka çalışır.**

1. Menu-driven input — kullanıcıdan geçerli seçim al:

```
do {
    printf("Seçim (1-3): ");
    scanf("%d", &choice);
} while (choice < 1 || choice > 3);
```

Önce seç, sonra valid mi diye bak. İlk turda da iste.

2. Newton iterasyonu — yakınsama testi:

```
do {
    x_new = (x + n/x) / 2;
    diff = fabs(x_new - x);
    x = x_new;
} while (diff > EPSILON);
```

En az bir iterasyon yapılmalı (diff hesabı için).

3. Buffer okuma — EOF'a kadar:

```
do {  
    n = read(fd, buf, BUFSIZE);  
    process(buf, n);  
} while (n > 0);
```

Read en az bir kez çağrılmalı (durumu bilebilmek için).

12. Speculate as to the reason control can be transferred into a C loop statement.

TR: *C'de bir loop'un içine kontrol transferi (goto ile dışarıdan loop body'sine atlama) niye mümkündür? Tahminin nedir?*

C'nin tasarım felsefesi ****"trust the programmer"****dı. ALGOL 60'ın da bu özelliği vardı; C bunu reddetmedi.

Olası gerekçeler:

1. **Tarihsel devamlılık:** ALGOL ve B'den miras. C kasıtlı olarak "minimum kısıtlama" felsefesindeydi.
2. **Implementation kolaylığı:** Compiler'ın goto'yu daha kısıtlı bir formla çözmesi (örn. sadece dışarı atlayan goto) ekstra analiz gerektirir. C compiler'larının ilk versiyonları minimal kaldı.
3. **Performans kaçıışı:** O dönemin manuel optimize edilmiş kodları için goto-based jumps kritikti. Compiler optimizeler bugünkü kadar iyi olmadığı için programcıya goto'yu açık bıraktılar.
4. **Düşük-seviye benzeri programlama:** C "yüksek-seviye assembler" olarak tasarlandı; assembly'de jump'ın hiçbir kısıtı yoktur, dolayısıyla C de izin verdi.

Pratik: Bu yetenek neredeyse hiç kullanılmaz; yapıldığında compiler optimization'ları bozar, debugger'lar şaşırır. Modern static analyzer'lar bu kalıpta uyarı verir. C99/C11'de hâlâ legal ama önerilmez.

Programming Exercises (9 soru)

1. Rewrite the following pseudocode segment using a loop structure in the specified languages:

```
k = (j + 13) / 27
loop:
  if k > 10 then goto out
  k = k + 1
  i = 3 * k - 1
  goto loop
out: . . .
```

a. C, C++, Java, or C# b. Python c. Ruby

TR: Yukarıdaki pseudokodu goto'suz, döngü yapısıyla yeniden yaz; sonra hangi dilin readability/writability avantajı olduğunu tartış.

(a) C / Java versiyonu:

```
int k = (j + 13) / 27;
while (k <= 10) {
  k = k + 1;
  i = 3 * k - 1;
}
// out: ...
```

(b) Python:

```
k = (j + 13) // 27
while k <= 10:
  k += 1
  i = 3 * k - 1
# out: ...
```

(c) Ruby:

```
k = (j + 13) / 27
while k <= 10
  k += 1
  i = 3 * k - 1
end
# out: ...
```

Değerlendirme:

- Yapı her dilde aynı `while` (pretest) — orijinal `goto`'lu kod tam olarak `while k <= 10` mantığını taşıyor.
- **Writability:** Python en kısa (`//` integer division, indentation).
- **Readability:** Üç dilde de eşit — basit bir `while` döngüsü.
- **Best combination:** Python burada öne çıkar — kısa, okunaklı, syntax noise yok.

2. Redo Programming Exercise 1, except this time make all the variables and constants floating-point type, and change `k = k + 1` to `k = k + 1.2`.

TR: Yukarıdaki problemi float olarak tekrarla; $k = k + 1.2$ ile.

```
double k = (j + 13.0) / 27.0;
while (k <= 10.0) {
  k = k + 1.2;
  i = 3.0 * k - 1.0;
}
```

Python ve Ruby'de tek fark: integer division `/` operatörünü kullan (Python `/` zaten float, `//` integer).

Önemli not: Float toplama precision hatası biriktirir. `k <= 10.0` testi `k = 9.999...` ile beklenmedik sonuç verebilir. Production kodda epsilon karşılaştırması düşünülmeli.

3. Rewrite the following code segment using a multiple-selection statement in the following languages:

```
if ((k == 1) || (k == 2)) j = 2 * k - 1
if ((k == 3) || (k == 5)) j = 3 * k + 1
if (k == 4) j = 4 * k - 1
if ((k == 6) || (k == 7) || (k == 8)) j = k - 2
```

a. C, C++, Java, or C# b. Python c. Ruby

TR: *Bu if-zincirini her dilin multiple selection yapısı ile yaz.*

(a) C / Java (switch):

```
switch (k) {
    case 1: case 2:
        j = 2 * k - 1; break;
    case 3: case 5:
        j = 3 * k + 1; break;
    case 4:
        j = 4 * k - 1; break;
    case 6: case 7: case 8:
        j = k - 2; break;
}
```

(b) Python — match (3.10+) veya zincirleme if:

```
match k:
    case 1 | 2:
        j = 2 * k - 1
    case 3 | 5:
        j = 3 * k + 1
    case 4:
        j = 4 * k - 1
    case 6 | 7 | 8:
        j = k - 2
```

Python 3.10 öncesinde dict mapping veya if/elif kullanılır:

```
if k in (1, 2):      j = 2 * k - 1
elif k in (3, 5):    j = 3 * k + 1
elif k == 4:         j = 4 * k - 1
elif k in (6, 7, 8): j = k - 2
```

(c) Ruby (case):

```
case k
when 1, 2
  j = 2 * k - 1
when 3, 5
  j = 3 * k + 1
when 4
  j = 4 * k - 1
when 6, 7, 8
  j = k - 2
end
```

Karşılaştırma:

- **Ruby** `when` virgülle çoklu değer — okunması en sezgisel.
- **Python** `match ( | )` — modern ve güçlü pattern matching.
- **C** `switch case ... case ...` (boş fall-through) — çalışıyor ama syntactic olarak en grotesk; her case'in `break` istemesi gürültü.

Bu örnek için **Ruby** en iyi; C/Java en zayıf.

4. Consider the following C program segment. Rewrite it using no `gotos` or `breaks`.

```
j = -3;
for (i = 0; i < 3; i++) {
    switch (j + 2) {
        case 3:
        case 2: j--; break;
        case 0: j += 2; break;
        default: j = 0;
    }
    if (j > 0) break;
    j = 3 - i;
}
```

TR: *Bu kodu goto ve break kullanmadan yeniden yaz. (Quiz 12A ile aynı soru.)*

```
j = -3;
int done = 0;
int i = 0;
while (i < 3 && !done) {
    int sel = j + 2;
    if (sel == 3 || sel == 2)
        j--;
    else if (sel == 0)
        j += 2;
    else
        j = 0; // default

    if (j > 0)
        done = 1;
    else
        j = 3 - i;

    i++;
}
```

Notlar:

- case 3: (boş, fall-through ediyor) case 2: ile beraber `sel == 3 || sel == 2` 'ye dönüştü.
- Switch break'leri zaten her case için zorunluydu — if/else if zincirinde otomatik olarak doğru davranır.
- Loop break'i (`if (j > 0) break;`) → `done` flag'i + `while` koşulu.
- `for` → `while` çevrildi, `i++` body sonuna taşındı.

5. In a letter to the editor of CACM, Rubin (1987) uses the following code segment as evidence that the readability of some code with gotos is better than the equivalent code without gotos. This code finds the first row of an n by n integer matrix named x that has nothing but zero values.

```
for (i = 1; i <= n; i++) {
    for (j = 1; j <= n; j++)
        if (x[i][j] != 0)
            goto reject;
    println('First all-zero row is:', i);
    break;
reject:
}
```

Rewrite this code without gotos in one of the following languages: C, C++, Java, or C#. Compare the readability of your code to that of the example code.

TR: Yukarıdaki goto'lu kodu goto'suz yaz ve readability karşılaştır.

Java labeled break ile:

```
outer:
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        if (x[i][j] != 0)
            continue outer;    // bu satırı atla, dış for'a geç
    }
    System.out.println("First all-zero row is: " + i);
    break outer;               // dış for'dan çık
}
```

Flag-based (klasik C):

```
int found = 0;
for (int i = 1; i <= n && !found; i++) {
    int row_all_zero = 1;
    for (int j = 1; j <= n; j++) {
        if (x[i][j] != 0) {
            row_all_zero = 0;
            break;          // (inner break; ama "no breaks" şartı sağlanmalı...)
        }
    }
    if (row_all_zero) {
        printf("First all-zero row is: %d\n", i);
        found = 1;
    }
}
```

Boolean-only, hiçbir break/goto ile (en saf):

```
int i = 1, found = 0;
while (i <= n && !found) {
    int j = 1, has_nonzero = 0;
    while (j <= n && !has_nonzero) {
        if (x[i][j] != 0)
            has_nonzero = 1;
        else
            j++;
    }
    if (!has_nonzero) {
        printf("First all-zero row is: %d\n", i);
        found = 1;
    } else {
        i++;
    }
}
```

Readability karşılaştırması:

Versiyon	Satır	Okunabilirlik
Orijinal (goto'lu)	~7	Yapı: "match if not bad row, else jump to next row" — algoritma direkt görünür
Java labeled	~8	continue outer niyet'i çok net

Versiyon	Satır	Okunabilirlik
Flag-based	~12	İki flag, ek değişkenler — kalabalık
Saf yapısal	~14	En uzun, en yoğun — algoritma gizlendi

Sonuç: Rubin'in iddiası — bu özel kalıpta goto **gerçekten** daha okunaklı. **Ama** Java'nın labeled break/continue'su benzer netlikte; goto'nun "vahşi sıçrama" tehlikesini taşımaz. Çağdaş yaklaşım: dilin sunduğu sınırlı kontrol-akışı araçlarını kullan, ham goto'ya başvurma.

6. Consider the following programming problem: The values of three integer variables — first, second, and third — must be placed in the three variables max, mid, and min, with the obvious meanings, without using arrays or user-defined or predefined subprograms. Write two solutions to this problem, one that uses nested selections and one that does not. Compare the complexity and expected reliability of the two.

TR: *first/second/third → max/mid/min (sıralama). Biri nested selection ile, biri olmadan. Complexity ve reliability karşılaştır.*

Çözüm 1 — Nested selection (iç içe if):

```
if (first >= second) {
    if (first >= third) {
        max = first;
        if (second >= third) { mid = second; min = third; }
        else { mid = third; min = second; }
    } else {
        max = third; mid = first; min = second;
    }
} else { // second > first
    if (second >= third) {
        max = second;
        if (first >= third) { mid = first; min = third; }
        else { mid = third; min = first; }
    } else {
        max = third; mid = second; min = first;
    }
}
```

Çözüm 2 — Sequential, en çok 3 swap ile (sort network):

```
max = first; mid = second; min = third;

int t;
// max ≥ mid
if (max < mid) { t = max; max = mid; mid = t; }
// max ≥ min
if (max < min) { t = max; max = min; min = t; }
// mid ≥ min
if (mid < min) { t = mid; mid = min; min = t; }
```

Karşılaştırma:

Kriter	Nested	Sequential (swap)
Satır sayısı	~15	~6 (+ açıklama)
Cyclomatic complexity	6 (her if dalı)	3 (üç bağımsız if)
Test path sayısı	6 (her permütasyon)	3'er bağımsız
Okuma kolaylığı	Zor — her dal aynı şeyi tekrar	Kolay — algoritma "bubble-like sort"
Hata yapma riski	Yüksek (6 dalın hepsini doğru yazmak)	Düşük (üç simetrik karşılaştırma)
Eşitlik durumu	Tüm dallarda manual	Doğal — < kullanımı ile garanti

Reliability: Sequential daha güvenilir. Nested'ı yazarken sık yapılan hata, eşitlik durumlarında (örn. first == second) yanlış branch'e atlanmak. Sequential 3 swap, sort network'ün matematiksel garantisini taşır — 3 eleman için her zaman doğru.

Sonuç: Yapısal olarak basit (linear) çözüm hem okunabilirlikte hem güvenilirlikte kazanır. Bu, "düşük cyclomatic complexity = az bug" prensibinin küçük ölçekli bir örneği.

7. Rewrite the C program segment of Programming Exercise 4 using if and goto statements in C.

TR: *Programming Exercise 4'teki kodu sadece if ve goto ile yaz.*

```

j = -3;
int i = 0;
top:
    if (i >= 3) goto end;
    int sel = j + 2;

    if (sel != 3 && sel != 2) goto check0;
    j--;
    goto after_switch;

check0:
    if (sel != 0) goto default_case;
    j += 2;
    goto after_switch;

default_case:
    j = 0;

after_switch:
    if (j > 0) goto end;
    j = 3 - i;
    i++;
    goto top;

end:
    // ...

```

Bu, derleyicinin yapısal yapıları derledikten sonra ürettiği akışın temsili. Okunabilirlik son derece düşük — bu, "structured constructs neden değerli" sorusunun direkt cevabı.

8. Rewrite the C program segment of Programming Exercise 4 in Java without using a switch statement.

TR: *Programming Exercise 4'teki C kodunu Java'da switch olmadan yaz.*


```

int j = -3;
boolean done = false;
for (int i = 0; i < 3 && !done; i++) {
    int sel = j + 2;
    if (sel == 3 || sel == 2) {
        j--;
    } else if (sel == 0) {
        j += 2;
    } else {
        j = 0;
    }

    if (j > 0) {
        done = true;
    } else {
        j = 3 - i;
    }
}

```

Java break'i de orijinal C versiyonunun loop-terminating break'i gibi kullanılabilirdi, ama burada `done` flag'i tercih edildi (daha "no goto" tarzında).

9. Translate the following call to Scheme's COND to C and set the resulting value to y.

```

(COND
  ((> x 10) x)
  ((< x 5) (* 2 x))
  ((= x 7) (+ x 10))
)

```

TR: *Bu Scheme COND ifadesini C'ye çevir ve sonucu y'ye ata.*

Önemli not: Scheme'de COND'un test'leri **sırayla** değerlendirilir; ilk `#t` veren branch'in `expr`'i döner. Diğerleri kontrol edilmez bile.

```

int y;
if (x > 10)
    y = x;
else if (x < 5)
    y = 2 * x;
else if (x == 7)
    y = x + 10;
else
    y = 0;    // Scheme'de hiçbir clause match etmezse undefined; C'de bir default şart

```

Detay: Scheme'in original COND'unda ELSE clause yok → hiçbir test true olmazsa sonuç **undefined**. C'de bunu temsil etmek için ya:

- y 'yi başlangıçta tanımsız bırakırsın (riskli),
- veya açıkça bir default (y = 0 gibi) atarsın,
- veya hata fırlatırsın (exit(1) veya bir flag).

x == 7 clause'unun **x > 10** clause'undan **sonra** test edildiğini unutma: x = 7 durumunda zaten ilk clause 7 > 10 = false , ikinci 7 < 5 = false , üçüncü 7 == 7 = true → y = 17 döner. Doğru.

Ama x = 20 durumunda **ilk clause yakalar** (y = 20); x = 7 'nin özel kontrolü zaten erişilmez.

Tasarımdaki bu sıralama bilinçli mi rastlantı mı, soruda açık değil — ama Scheme semantiği "ilk match kazanır" olduğu için C'de else-if zinciri tam karşılığıdır.

Chapter 15 — Functional Programming Languages

Not: Bu bölümde 50 review question var ama büyük çoğunluğu ML, Haskell, F#, Common Lisp özelinde. Bizim final ağırlığımız **Scheme**'de (Quiz 11A). Scheme'le ilgili olanları detaylı, diğerlerini kısa çözüyorum.

Review Questions (50 soru)

1. Define functional form, simple list, bound variable, and referential transparency.

TR: *Functional form, simple list, bound variable ve referential transparency'yi tanımla.*

- **Functional form (yüksek-mertebeli işlev):** Bir veya daha fazla fonksiyonu argüman alan, ya da fonksiyon döndüren bir fonksiyon. Örn: function composition operatörü $\circ$, ya da Scheme'in `MAP` 'i.
- **Simple list:** İçinde başka liste içermeyen, sadece atomlardan oluşan liste. `(a b c)` simple, `(a (b c) d)` simple değil.
- **Bound variable:** Bir lambda expression veya let bağlamında bağlanmış değişken — bir değer atanmış, "free" olmayan değişken.
- **Referential transparency:** Bir ifadeyi değeriyle değiştirebilme özelliği — program davranışı değişmez. Side-effect'siz fonksiyonlarda mümkündür. Saf functional dilin tanımlayıcı özelliği.

2. What does a lambda expression specify?

TR: *Bir lambda ifadesi ne belirtir?*

İsimsiz bir fonksiyonu — parametrelerini ve gövdesini. Scheme:

```
(LAMBDA (x) (* x x)) ; "x al, x*x döndür"
```

Lambda'lar değer gibi davranır: bir değişkene atanabilir, başka fonksiyona geçirilebilir, dönüş olarak verilebilir.

3. What data types were parts of the original Lisp?

TR: *Orijinal Lisp'in veri tipleri neydi?*

İki tip: **atom** ve **list**.

- Atom: sayılar, semboller (ad).
- List: sıfır veya daha fazla eleman içeren, parantezli yapı (boş liste `()` dahil).

Bu minimalizm Lisp'in gücüydü — her şey atom veya liste; kod kendisi de bir liste (homoiconicity).

4. In what common data structure are Lisp lists normally stored?

TR: *Lisp listeleri normalde hangi veri yapısında saklanır?*

Cons cell linked list. Her cons cell iki pointer içerir: `CAR` (head'e) ve `CDR` (tail'e). Boş liste özel bir sentinel `('() / nil)`.

$(a\ b\ c) \equiv [a|\bullet] \rightarrow [b|\bullet] \rightarrow [c|\bullet] \rightarrow \text{nil}$
 $\uparrow$ car $\uparrow$ cdr ...

Bu yapı $O(1)$ head erişimi ve $O(1)$ eleman ekleme (CONS) sağlar; eleman sayısı $O(n)$.

5. Explain why QUOTE is needed for a parameter that is a data list.

TR: *Data liste parametresi için QUOTE niye gerek?*

Scheme'in evaluation kuralı: bir liste $(e_1\ e_2\ \dots\ e_n)$ ise, e_1 fonksiyon kabul edilir ve $e_2..e_n$ argüman olarak değerlendirilir. Yani $(a\ b\ c)$ deyince, Scheme a 'yı fonksiyon sanıp b ve c ile çağırmak ister.

Eğer liste sadece **veri** olarak kullanılacaksa (fonksiyon çağırısı değil), bunu Scheme'e söylemek gerekir — QUOTE veya kısayolu `'` :

`'(a b c)` ; literal liste – değerlendirilmez
`(QUOTE (a b c))` ; aynı şey

Olmazsa: $(a\ b\ c) \rightarrow$ "a fonksiyonunu b ve c ile çağır" $\rightarrow$ muhtemelen error.

6. What is a simple list?

TR: *Simple list nedir?*

Soru 1'de tanımlandı: yalnızca atomlardan oluşan, alt-liste içermeyen liste. $(a\ b\ c)$ simple, $(a\ (b)\ c)$ değil.

7. What does the abbreviation REPL stand for?

TR: *REPL kısaltması ne anlama gelir?*

Read-Eval-Print Loop. Interactive bir interpreter'ın çalışma döngüsü:

1. **Read:** Kullanıcıdan ifade oku.
2. **Eval:** İfadeyi değerlendir.
3. **Print:** Sonucu bas.
4. **Loop:** Başa dön.

Lisp/Scheme dünyasının icadı; Python, Ruby, Node.js gibi modern dillerde de standart.

8. What are the three parameters to IF?

TR: *IF'in üç parametresi nedir?*

(IF cond then-expr else-expr) — test (Boolean), true-expr, false-expr.

Scheme'de IF bir **special form** (özel ifade) — tüm üç parametre değerlendirilmez. Önce cond, sonra sadece bir branch.

9. What are the differences between =, EQ?, EQV?, and EQUAL?

TR: *=, EQ?, EQV?, EQUAL? farkları nelerdir?*

Operatör	Çalışır mı	Tip kısıtı	Detay
=	Sayılar	Yalnız sayı	Aritmetik eşitlik. (= 3 3.0) → #t (Scheme implementation-dependent)
EQ?	Atomlar	Sembol / boolean / char	"Aynı bellek nesnesi mi?" — tam kimlik. Sayılarda güvensiz, sembollerde güvenilir.
EQV?	Atomlar (sayılar dahil)	Atomlar	EQ?'in geneli; sayıları değer olarak da karşılaştırır.
EQUAL?	Her şey	Tüm tipler	Yapısal/derin karşılaştırma. Listelerde içerik eleman eleman aynı mı?

Pratikte:

- Sembol: EQ? veya EQUAL? .
- Sayı: = veya EQV? .
- Liste: EQUAL? .

10. What are the differences between the evaluation method used for the Scheme special form **DEFINE** and that used for its primitive functions?

TR: *Scheme'in DEFINE special form'u ve primitive fonksiyonları farklı nasıl değerlendirilir?*

Primitive fonksiyon çağrısı: Tüm argümanlar önce değerlendirilir, sonra fonksiyon çağrılır. Bu **applicative-order** (eager).

DEFINE (special form): "Special" olmasının nedeni — argümanları normal şekilde değerlendirmez.

(DEFINE x (+ 2 3)) deyince:

- x (sembol) değerlendirilmez — olduğu gibi değişken adı olarak kullanılır.
- (+ 2 3) değerlendirilir $\rightarrow 5$.
- x sembolüne 5 değeri bind edilir.

Eğer DEFINE primitive olsaydı, x 'i önce değerlendirmeye çalışırdı $\rightarrow$ x bound değil $\rightarrow$ error.

Aynı argüman LAMBDA, IF, COND, QUOTE için de geçerli — hepsi special form, parametrelerini "normal" değerlendirmezler.

11. What are the two forms of DEFINE?

TR: *DEFINE'in iki formu nedir?*

Form 1 — Değişken (variable binding):

```
(DEFINE name expression)
```

Örn: (DEFINE pi 3.14159) .

Form 2 — Fonksiyon (LAMBDA syntactic sugar):

```
(DEFINE (fn-name params) body)
```

Örn: (DEFINE (square x) (* x x)) $\equiv$ (DEFINE square (LAMBDA (x) (* x x))) .

12. Describe the syntax and semantics of COND.

TR: *COND'un söz dizimi ve anlam bilgisini açıkla.*

Syntax:

```
(COND
  (test1 expr1)
  (test2 expr2)
  ...
  (ELSE default-expr))
```

Semantics:

1. test1 değerlendir.
2. #t (true) ise expr1 değerlendir, sonucu döndür. Diğer clause'lara bakma.
3. #f ise test2 'ye geç, tekrarla.
4. Hiçbiri true değil ve ELSE clause yok → return değeri **undefined**.
5. ELSE clause varsa onun expr'i sonuçtur.

Bu, multi-way selection için temel yapıdır. C/Java'nın if-else if-else if-else zincirine eşdeğer.

13. Why are CAR and CDR so named?

TR: *CAR ve CDR niye böyle isimlendi?*

Tarihsel sebep — orijinal Lisp IBM 704 makinesinde uygulandı. IBM 704'ün 36-bit kelime yapısında her cons cell:

- **CAR** = "Contents of Address Register" — kelimenin address kısmı (head pointer)
- **CDR** = "Contents of Decrement Register" — kelimenin decrement kısmı (tail pointer)

Donanım terminolojisi 1958'de tutsa da, isimler bugüne kaldı. Bazı modern Lisp/Scheme dialektleri `FIRST` ve `REST` alias'larını da sağlar.

14. If CONS is called with two atoms, say 'A and 'B, what is the returned?

TR: *CONS iki atomla çağrılırsa (örn 'A ve 'B) ne döner?*

(CONS 'A 'B) → (A . B) — bir **dotted pair** (cons-pair). Bu **liste DEĞİL** çünkü düzgün listenin son CDR'ı '() olmalı.

Karşılaştırma:

- (CONS 'A 'B) → (A . B) — pair
- (CONS 'A '()) → (A) — tek elemanlı liste (düzgün liste)
- (CONS 'A '(B)) → (A B) — iki elemanlı düzgün liste
- (CONS 'A (CONS 'B '())) → (A B) — el ile inşa

15. Describe the syntax and semantics of LET in Scheme.

TR: *Scheme'de LET'in söz dizimi ve anlam bilgisini açıkla.*

Syntax:

```
(LET ((var1 expr1)
      (var2 expr2)
      ...))
  body)
```

Semantics:

1. Tüm `expr_i` **paralel** değerlendirilir (`var_j`'leri henüz görmeden — birbirini referans alamazlar).
2. `var_1...var_n` yeni bir scope'ta bind edilir.
3. `body` bu scope'ta değerlendirilir.
4. Sonuç `body`'nin sonucudur.

```
(LET ((a 3) (b 4))
      (+ (* a a) (* b b)))    ; → 25
```

Varyantlar:

- LET* — sıralı (sonraki, öncekini görür).
- LETREC — recursive (kendisini görür; mutual recursion için).

16. What are the differences between CONS, LIST, and APPEND?

TR: *CONS, LIST ve APPEND arasındaki farklar.*

(CONS 'a '(b c)) → (a b c) ; başa BİR eleman ekle
 (LIST 'a 'b 'c) → (a b c) ; n eleman al, liste yap
 (APPEND '(a b) '(c d)) → (a b c d) ; iki listeyi birleştir

	Kullanım
CONS	İlk argüman = atom/value, ikinci = liste → başa ekleme.
LIST	n argüman, hepsi listenin elemanı olur.
APPEND	Argümanlar liste olmak zorunda, hepsini birleştirir.

Karışıklık tuzağı:

- (CONS '(a b) '(c d)) → ((a b) c d) — ilk argüman BİR ELEMEN olarak eklenir.
- (APPEND '(a b) '(c d)) → (a b c d) — ilk argümanın elemanları açılır.

17. Describe the syntax and semantics of mapcar in Scheme.

TR: Scheme'de mapcar'ın söz dizimi ve anlam bilgisini açıkla.

Aslında Scheme'de adı MAP . (Common Lisp'te mapcar .)

Syntax: (MAP function list1 list2 ...)

Semantics: function'ı, list1...listN'nin paralel elemanlarına uygular. Sonuç, function'ın her uygulamasının sonuçlarının listesi.

(MAP (LAMBDA (x) (* x x)) '(1 2 3 4)) → (1 4 9 16)
 (MAP + '(1 2 3) '(10 20 30)) → (11 22 33)

İki ya da daha fazla liste varsa, function de o kadar argüman kabul etmelidir.

18. What is tail recursion? Why is it important to define functions that use recursion to specify repetition to be tail recursive?

TR: Tail recursion nedir? Yinelemeyi recursion ile yazmak istediğimizde fonksiyonu tail recursive yapmak niye önemli?

Tail recursion: Bir fonksiyonun recursive çağrısı, fonksiyonun **son işlemi** ise (çağrıdan sonra başka iş yapılmıyorsa). Örnek:

```
; TAIL recursive
(DEFINE (fact-helper n acc)
  (IF (= n 0) acc
      (fact-helper (- n 1) (* acc n)))) ; çağrı SON

; NOT tail recursive – çağrıdan sonra '* n' var
(DEFINE (fact n)
  (IF (= n 0) 1
      (* n (fact (- n 1)))))
```

Önemi:

- Scheme standardı tail recursion'ı **TCO (Tail Call Optimization)** ile mecbur tutar — fonksiyon stack frame'i geri kullanılır.
- Yani sabit stack boyutuyla çalışır → büyük input'larda **stack overflow olmaz**.
- Imperative loop'un performans karakteristiğine ulaşır.

Functional dillerin "recursion yerine loop yok" tasarımı, tail recursion + TCO olmadan pratik olmazdı.

19. Why were imperative features added to most dialects of Lisp?

TR: *Lisp'in çoğu lehçesine imperative özellikler niye eklendi?*

Pragmatik gerekçeler:

1. **Performans:** Pure functional yaklaşım her veri yapısı değişikliğinde yeni kopya yaratır. Imperative mutation (set!, vector-set!) çok daha hızlı.
2. **I/O yönetimi:** Dosya, terminal, network gibi yan etkili işlemleri saf functional ifade etmek zor (monad'lar olmadan).
3. **Mevcut bilgi birikimi:** Programcılar imperative düşünmeye alışkın; loop, atama, side-effect tanıdık.
4. **Sistem programlama:** Düşük-seviye nesne yönetimi için mutation kaçınılmaz.
5. **Veri yapıları efficiency:** Hash table, queue gibi yapıların verimli implementation'ı için in-place değişim gerekir.

Bunun bedeli: Referential transparency kaybı, optimization güçlüğü, paralellik zorluğu. Haskell bu nedenle baştan **saf** kalmayı seçti; Lisp/Scheme pragmatizm tercih etti.

20. In what ways are Common Lisp and Scheme opposites?

TR: *Common Lisp ve Scheme hangi açılardan birbirinin zıttıdır? (kısa)*

	Common Lisp	Scheme
Boyut	Devasa, "kitchen sink"	Minimal, "purist"
Standardı	ANSI 1994, çok geniş	R5RS/R6RS/R7RS, küçük
Scoping	Optional dynamic (default static)	Yalnız static (lexical)
Multiple return	Var (values)	Standartta yok (continuation ile)
Object system	CLOS (zengin)	Yok (kütüphane)
Numeric tower	Karmaşık	Tutarlı, basit

Felsefe: Common Lisp "gerekli her şeyi sağla"; Scheme "küçük çekirdek, gerisini kullanıcı yazsın".

21. What scoping rule is used in Scheme? In Common Lisp? In ML? In Haskell? In F#?

TR: *Hangi dilde hangi scoping kuralı?*

Hepsi **static (lexical) scoping** kullanır. Common Lisp default static, ama `(declare special)` ile dynamic scoping da mümkündür.

22. What happens during the reader phase of a Common Lisp language processor?

TR: *Common Lisp processor'un reader fazında ne olur? (kısa)*

Karakter akışını alır, içsel S-expression (list/atom yapısı) hâline çevirir. Bu, parse'in ilk aşaması; sonraki aşama macroexpansion, sonra evaluation.

23. What are two ways that ML is fundamentally different from Scheme?

TR: *ML, Scheme'den temelde nasıl farklı? (kısa)*

1. **Static typing + type inference:** ML her ifadenin tipini compile-time'da çıkarır. Scheme dynamic.
2. **Pattern matching:** ML fonksiyonları case'lerle (pattern) tanımlanır. Scheme COND/IF zinciri.

24. What is stored in an ML evaluation environment?

TR: *ML evaluation environment'ında ne saklanır? (kısa)*

Bind edilmiş değişkenler ve fonksiyonlar — name → value eşlemesi. ML'in immutable yapısından ötürü "environment" değişmez; yeni binding'ler yeni environment yaratır.

25. What is the difference between an ML val statement and an assignment statement in C?

TR: *ML val ile C atama farkı? (kısa)*

- C: `x = 5;` mutable — değişkenin hücresinin değerini değiştirir.
- ML: `val x = 5;` immutable — `x` adına 5 değerini bind eder; yeni bir `val x = 7` öncekini ÖRTÜR (shadow), ama eski binding hâlâ var (capture eden closure'lar görür).

26. What is type inferencing, as used in ML?

TR: *ML'de type inference nedir? (kısa)*

Tip anotasyonu olmadan, ifadenin tipini context'ten otomatik çıkaran sistem. Örn. `fun double x = x * 2` yazınca, `*` `int` için tanımlı → ML `x: int → int` olduğunu bulur. Hindley-Milner type system.

27. What is the use of the fn reserved word in ML?

TR: *ML'de fn nedir? (kısa)*

Lambda expression. Scheme'in LAMBDA'sına karşılık: `fn x => x * 2`.

28. Can ML functions that deal with scalar numerics be generic?

TR: *ML'in numeric fonksiyonları generic olabilir mi?* (kısa)

Tam olarak değil — ML int ve real için ayrı `+`, `*` operatörleri kullanır. Generic numeric tower yok.

Workaround: type classes (Haskell'de var) veya overloading.

29. What is a curried function?

TR: *Curried function nedir?*

Çoklu argüman alan bir fonksiyonu, her seferinde TEK argüman alan iç içe fonksiyonlar dizisi olarak gerçekleyen yapı. Haskell B. Curry'den.

```
(* normal *)
fun add(x, y) = x + y;      (* tip: int * int → int *)

(* curried *)
fun add x y = x + y;      (* tip: int → int → int = int → (int → int) *)

add 3                      (* int → int, "üç ekleme fonksiyonu" *)
add 3 4                    (* 7 *)
```

Avantaj: **partial application** — fonksiyonu ilk argümanla "sabitle", kalan argümanlar için yeni fonksiyon al.

```
val addThree = add 3      (* "3 ekleme" fonksiyonu *)
addThree 10                (* 13 *)
```

30. What does partial evaluation mean?

TR: *Partial evaluation ne anlama gelir?* (kısa)

Bir fonksiyonu, bazı argümanları sabit verip kısmen değerlendirmek; sonuç, kalan argümanları alan yeni bir fonksiyondur. Curried fonksiyonlarda doğal.

31. Describe the actions of the ML filter function.

TR: *ML filter fonksiyonu ne yapar?* (kısa)

Bir predicate ve liste alır; predicate'i `true` döndüren elemanlardan oluşan yeni liste döndürür.

```
filter (fn x => x > 0) [-1, 2, -3, 4]    (* [2, 4] *)
```

32. What operator does ML use for Scheme's CAR?

TR: *ML'de Scheme'in CAR'ının karşılığı?* (kısa)

`hd` (head). `tl` (tail) ise CDR. Pattern matching ile genelde doğrudan `[h::t]` syntax'i kullanılır.

33. What operator does ML use for functional composition?

TR: *ML'de fonksiyon kompozisyonu operatörü?* (kısa)

`o` (küçük o harfi). $(f \circ g) x \equiv f (g x)$.

34. What are the three characteristics of Haskell that make it different from ML?

TR: *Haskell'i ML'den ayıran üç özellik?* (kısa)

1. **Lazy evaluation** (default). ML strict.
2. **Pure** — side effect yok (I/O monad'larla); ML imperative özellikler içerir.
3. **Type classes** (overloading mekanizması). ML'de yok.

35. What does lazy evaluation mean?

TR: *Lazy evaluation nedir?*

İfadeler **ihtiyaç olana kadar değerlendirilmez**. Argümanlar fonksiyona geçirilirken hesaplanmaz, fonksiyon içinde gerçekten kullanıldıkları yerde değerlendirilir.

Avantajları:

- Sonsuz veri yapıları mümkün: [1..] Haskell'de tüm doğal sayılar.
- Gereksiz hesaplama yok.

Dezavantajı:

- Bellek profili tahmin edilemez (thunks birikir).
- Debug zor — "hangi sırayla hesaplandı?" net değil.

36. What is a strict programming language?

TR: *Strict bir dil nedir? (kısa)*

Lazy'nin tersi — argümanlar fonksiyona geçirilmeden önce değerlendirilir (eager evaluation). C, Java, Python, ML, Scheme strict. Haskell lazy.

37. What programming paradigms are supported by F#?

TR: *F# hangi paradigmaları destekler? (kısa)*

Functional (primary), object-oriented, imperative. .NET üzerinde çalışır → .NET ekosistemiyle entegre.

38. With what other programming languages can F# interoperate?

TR: *F# hangi dillerle birlikte çalışabilir? (kısa)*

Tüm .NET dilleri: C#, [VB.NET](#), IronPython, vs. CLR üzerinden tek bir runtime.

39. What does F#'s let do?

TR: *F#'in let'i ne yapar? (kısa)*

Bir değişkeni (veya fonksiyonu) bir scope içinde tanımlar/bind eder. Default immutable; `let mutable` ile değiştirilebilir hâle gelir.

40. How is the scope of a F# let construct terminated?

TR: *F#'in let scope'u nerede biter?* (kısa)

Girintilemeye göre. Aynı veya daha az girintili sonraki ifade let'in scope'unu sonlandırır.

41. What is the underlying difference between a sequence and a list in F#?

TR: *F#'te sequence ile list farkı?* (kısa)

- **List:** Eager, sonlu, immutable. Tüm elemanlar bellekte.
- **Sequence (seq):** Lazy, potansiyel sonsuz, elemanlar talep edildikçe üretilir. IEnumerable benzeri.

42. What is the difference between the let of ML and that of F#, in terms of extent?

TR: *ML let ve F# let extent farkı?* (kısa)

ML'de `let in expr` bağlaması yalnız `expr` içinde geçerli — limited extent. F#'te girintiye dayalı, daha uzun scope'lara genişler. F# let, top-level olabilir; ML'de top-level binding `val` ile yapılır.

43. What is the syntax of a lambda expression in F#?

TR: *F#'te lambda söz dizimi?* (kısa)

```
fun x -> x * 2
fun x y -> x + y
```

44. Does F# coerce numeric values in expressions? Argue in support of the design choice.

TR: *F# numeric coercion yapar mı? Tasarım kararını destekle.* (kısa)

Hayır, yapmaz. `int + float` derleme hatası. Explicit cast şart.

Destek argümanı: Strong typing'in temel ilkesi — implicit coercion bug kaynağı. Numeric precision kaybı (float → int truncate), overflow gibi sorunlar erken yakalanır. F#'in formal-yöntem ağırlıklı kullanım profili için bu kritik.

45. What support does Python provide for functional programming?

TR: *Python functional programlama için ne sağlar? (kısa)*

- lambda ifadeleri
- map , filter , reduce (functools)
- List/dict/set comprehensions
- Generators (yield) — lazy iteration
- First-class functions
- functools.partial — partial application

Saf functional dil değildir, ama hibrid kullanım yaygın.

46. What function in Ruby is used to create a curried function?

TR: *Ruby'de curried function nasıl oluşturulur? (kısa)*

Proc#curry veya Method#curry :

```
add = lambda { |x, y| x + y }  
add_curried = add.curry  
add5 = add_curried[5]  
add5[3]    # 8
```

47. Is the use of functional programming expanding or shrinking?

TR: *Functional programlama kullanımı genişliyor mu daralıyor mu? (kısa)*

Genişliyor. Mainstream diller (Java, C#, JavaScript, Swift) functional özellikler ekledi (lambda, stream, immutable veri yapıları). Saf functional diller (Haskell, F#, Clojure, Elm) niş ama büyüyor.

Concurrent/parallel programming gereksinimi functional yaklaşımı zorunlu kılıyor.

48. What is one characteristic of functional programming languages that makes their semantics simpler than that of imperative languages?

TR: *Functional dillerin semantiğini imperative dillerinden basit kılan özellik?*

Referential transparency + side-effect olmaması. Bir ifadenin değeri sadece argümanlarına bağlı; çağrı sırası, global state, çevre değişkenleri etkilemez. Bu, semantik analiz, eşdeğerlik kanıtı, optimization için çok daha rahat bir çerçeve sağlar.

49. What is the flaw in using lines of code to compare the productivity of functional languages and that of imperative languages?

TR: *Functional vs imperative üretkenliği LOC ile karşılaştırma hatası?*

LOC (lines of code) abstraksiyon seviyesini ölçmez. Functional bir dilde tek satır map/filter, imperative'de 10 satır loop edebilir. Bu LOC karşılaştırması functional'ı **daha üretken** gösterse de, asıl ölçü "iş başına geçen süre" ve "bug oranı"dır. LOC tek başına yanıltır.

50. Why can concurrency be easier with functional languages than imperative languages?

TR: *Concurrency niye functional dillerle daha kolay?*

Mutable shared state yok. Imperative dillerin baş belası race condition (iki thread aynı değişkene yazar) → kilit, mutex, ... gerek. Functional'da değer immutable; iki thread aynı veriye baktığında çakışma imkânsız. Side effect yok → reordering güvenli. Erlang, Clojure, Haskell concurrency için bu avantajdan faydalanır.

Problem Set (10 soru)

1. Read John Backus's paper on FP (Backus, 1978) and compare the features of Scheme discussed in this chapter with the corresponding features of FP.

TR: Backus 1978 FP makalesini oku ve FP'nin özelliklerini Scheme'le karşılaştır.

Backus 1978: "Can Programming Be Liberated from the von Neumann Style?"

Backus'un FP (Functional Programming) dili akademik bir tasarım önerisidir. Pratik kullanılmadı ama Haskell ve modern functional dillere ilham verdi.

Özellik	FP	Scheme
Programlar	Fonksiyon kompozisyonu zincirleri	Lambda ifadeleri ve recursion
Veri	Atomlar + sequences	Atomlar + lists
Functional forms	Built-in ($\circ$, α , $/$, etc.)	LAMBDA, MAP, fold ile manuel
Değişken	YOK (point-free style)	Var (bind edilir)
Recursion	Combinators ile (Y)	Direkt recursion + LETREC
Verilerin türü	Algebraic structure (set with operations)	Cons-cell, atom
Side effect	Hiç	İzin var (set!)

Anahtar fark: FP **point-free** (değişkensiz) tasarımıdır — fonksiyonlar her zaman noktalı argümanlar ($f(x)$) yerine, fonksiyon kompozisyonu ($f \circ g$) ile inşa edilir. Scheme bind ve LAMBDA kullanır $\rightarrow$ değişken adıyla referans.

2. Find definitions of the Scheme functions EVAL and APPLY, and explain their actions.

TR: Scheme'in EVAL ve APPLY fonksiyonlarını bul, ne yaptıklarını açıkla.

EVAL : Bir veri yapısını (genelde quote'lanmış liste) alıp **Scheme ifadesi olarak değerlendirir**.

(EVAL '(+ 1 2) (the-environment)) $\rightarrow$ 3

(EVAL '(* 3 4) (the-environment)) $\rightarrow$ 12

EVAL, "kod = veri" prensibinin metalinguistic gücüdür — runtime'da yeni kod inşa edip çalıştırabilirsin.

APPLY : Bir fonksiyonu, bir liste olarak verilmiş argümanlarla çağırır.

```
(APPLY + '(1 2 3 4)) → 10
```

```
(APPLY MAX '(7 3 9 5)) → 9
```

APPLY = "fonksiyon + argüman listesi'ni unpack edip çağır". Higher-order kalıplarda kritik — örn. fonksiyon türetilirken argüman sayısı bilinmiyorsa APPLY kurtarır.

İkisi birlikte: **meta-circular evaluator** (kendi kendisini değerlendiren interpreter) FP'nin kalbidir; her ikisi de fundamental.

3. One of the most modern and complete programming environments is the INTERLISP system for Lisp. ... Read this article and compare ...

TR: *INTERLISP makalesini oku, kendi sisteminle karşılaştır.* (kısa)

INTERLISP (Xerox PARC, 1970'ler) — DWIM (Do What I Mean) ile typo'ları otomatik düzelten, structure editor (text değil yapısal düzenleme), tracer, masterscope (cross-reference) gibi modern IDE'lerin atalarını barındırıyordu.

Modern karşılaştırma:

- INTERLISP'in **structure editor**'ü bugün hâlâ yaygın değil (Lisp paren-sensitive bir dil olmasına rağmen).
- DWIM benzeri auto-correct yok (linter'lar uyarır ama "düzeltmez").
- REPL + cross-reference + debugger entegrasyonu bugünün IDE'lerinde standart (Emacs SLIME, DrRacket).

INTERLISP'i bugünkü Scheme/Racket IDE'leriyle karşılaştırırsan: yapısal özellikler oradaydı, ama paketleme/dağıtım/community ekosistemi modern hâl çok daha güçlü.

4. Refer to a book on Lisp programming and determine what arguments support the inclusion of the PROG feature in Lisp.

TR: *PROG özelliği niye Lisp'e eklendi?*

PROG : Klasik Lisp'in imperative escape valve'ı. İçinde:

- Lokal değişkenler bildirilir.
- Etiketler (label) konulur.
- GO ile goto yapılır.
- SETQ ile mutation, RETURN ile çıkış.

Argümanlar:

1. **Imperative algoritmalar:** Bazı algoritmalar (örn. union-find, in-place sort) doğal olarak imperative. PROG bunları zarif yazmaya izin verdi.
2. **Performance:** Saf recursion + functional kopyalama yavaştır. PROG mutation ile büyük veri yapıları üzerinde verimli işlem.
3. **I/O:** Imperative side-effect zorunlu; PROG bunu ifade için doğal yapı.
4. **Köprü kuruyordu:** Fortran/Algol kullanıcıları için tanıdık geliyor → adaptasyonu kolaylaştırıyor.

Modern bakış: PROG eleştirildi (Dijkstra'nın goto saldırısı ile aynı dönemde) ve sonradan Scheme'de DO ve LET ile değiştirildi. Common Lisp PROG'u koruyor ama modern Lisp'çiler nadiren kullanır.

5. Find at least one example of a typed functional programming language being used to build a commercial system in each of the following areas: database processing, financial modeling, statistical analysis, and bioinformatics.

TR: *Tipli functional dilin ticari kullanım örnekleri.*

- **Database processing:** Twitter'ın query engine'i bir dönem Scala (typed functional + OO). Microsoft SQL Server'ın bazı bileşenleri F# ile prototiplenmişti.
- **Financial modeling:** Jane Street Capital — yüksek-frekanslı trading'in tamamını **OCaml**'da yazıyor. Standard Chartered ve Barclays Haskell kullandı.
- **Statistical analysis:** R aslında dynamic ama functional ağırlıklı; Microsoft'un Solver Foundation F#. Python pandas+functools her yerde.
- **Bioinformatics:** Haskell'da BiGNoR, Bio.Hs gibi kütüphaneler. OCaml ile MOSAIK (DNA sequence analysis).

Genel trend: Functional dilin **tip güvenliği + concurrency + matematiksel netlik** kombinasyonu, finans ve veri bilimi gibi "yanlış cevap pahalı" alanlarda öne çıkıyor.

6. A functional language could use some data structure other than the list. For example, it could use strings of single-character symbols. What primitives would such a language have in place of the CAR, CDR, and CONS primitives of Scheme?

TR: *Liste yerine string kullanan bir functional dil olsa, CAR/CDR/CONS yerine hangi primitiveler olurdu?*

String'ler için karşılıklar:

- **CAR** → first-char veya head — ilk karakteri verir.
- **CDR** → rest-of-string veya tail — ilk karakterden sonrası.
- **CONS** → prepend-char — başa bir karakter ekler, yeni string döner.

Ek olarak gerekli:

- **Empty test:** (empty? s) — Scheme NULL? 'ın karşılığı.
- **Empty constructor:** boş string sabitliği ("").

Bu dilin diğer primitive'leri:

- Karakter karşılaştırma (atom karşılaştırma karşılığı): (char=? c1 c2) .
- Karakter atomu sinama: (char? x) .

Algoritmalar tam aynı pattern'i kullanır:

```
(DEFINE (length s)
  (IF (empty? s) 0
      (+ 1 (length (rest-of-string s)))))
```

Pratikte: Common Lisp'in string 'leri ve Haskell'in String = [Char] zaten bu fikrin pragmatik karışımıdır.

7. Make a list of the features of F# that are not in ML.

TR: *F#'in ML'de olmayan özellikleri. (kısa)*

1. **.NET ekosistemi entegrasyonu** — C#/VB.NET kütüphaneleri.
2. **Object-oriented yetenekler** — class, interface, inheritance.
3. **Active patterns** — gelişmiş pattern matching.
4. **Computation expressions** — async, query gibi monadik yapılara dilsel destek.
5. **Type providers** — runtime'da dış data kaynağından tip üretme.
6. **Mutable by default control** — let mutable ile değiştirilebilir bağlama.

7. **Sequence expressions** — lazy `seq { ... } .`

8. **Unit of measure** — fiziksel birimleri tipte taşıma.

ML'de olmayan ama F#'in 2.0+ ile geldikleri çoğunlukla.

8. If Scheme were a pure functional language, could it include **DISPLAY**? Why or why not?

TR: *Scheme saf functional olsaydı DISPLAY (yazdırma) içerebilir miydi?*

Hayır. `DISPLAY` bir **side effect** üretir (terminal/dosyaya yazar). Pure functional dilin tanımı: aynı argümanlarla aynı sonuç + side effect yok. `DISPLAY`:

- Argümanlar aynı olsa da ekrana farklı kez yazar (her çağrıda yeni bir yan etki).
- Dönüş değeri genelde `unspecified` — kullanılmıyor; çağrılma sebebi side effect.
- Referential transparency'yi yıkar.

Saf functional dilde alternatif:

- I/O monadlarıyla (Haskell'in IO monad'ı gibi) — side effect'i tip sisteminde işaretle.
- Veya prosedürün dönüş değerini "yazdırılacak şeylerin listesi" olarak modelle (Lisp'in `print` programları string biriktirir, main bunları çıktıya verir).

Scheme bilerek pragmatik — `DISPLAY` var ama "side effect içerdiğini bil" notu ile.

9. What does the following Scheme function do?

```
(define (y s lis)
  (cond
    ((null? lis) '() )
    ((equal? s (car lis)) lis)
    (else (y s (cdr lis)))))
```

TR: *Bu Scheme fonksiyonu ne yapar?*

Cevap: `member` 'in bir varyantı — `s` atomunun listede ilk geçtiği konumdan itibaren olan sub-liste'sini döndürür. Yoksa `'()` (boş liste).

Adım adım:

- `lis` boşsa $\rightarrow$ boş liste döndür.
- `lis` 'in ilk elemanı `s` ise $\rightarrow$ `lis` 'in tamamını döndür (`s` 'in olduğu noktadan itibaren).
- Değilse $\rightarrow$ (`cdr lis`) ile recurse.

Örnek:

```
(y 'c '(a b c d e))  → (c d e)
(y 'a '(a b c))      → (a b c)
(y 'x '(a b c))      → ()
```

Bu Scheme standardının `member` fonksiyonuyla **birebir aynı davranış**tadır (Scheme'in standart `member` 'ı da listenin geri kalanını verir, sadece `#t` / `#f` değil).

10. What does the following Scheme function do?

```
(define (x lis)
  (cond
    ((null? lis) 0)
    ((not (list? (car lis)))
     (cond
       ((eq? (car lis) #f) (x (cdr lis)))
       (else (+ 1 (x (cdr lis))))))
    (else (+ (x (car lis)) (x (cdr lis))))))
```

TR: *Bu Scheme fonksiyonu ne yapar?*

Cevap: Bir liste (nested olabilir) içindeki, atom olan ama `#f` (false) olmayan elemanların toplam sayısını döndürür. Yani **non-#f atom sayımı**, derinlemesine arama.

Mantık analizi:

- `lis` boşsa $\rightarrow 0$.
- (`car lis`) liste DEĞİLSE (yani atomsa):
 - Atom `#f` ise $\rightarrow$ bu eleman sayılmaz, kuyrukta devam et.
 - Atom başka bir şeyse $\rightarrow 1 +$ kuyruktaki sayım.
- (`car lis`) listeyse $\rightarrow$ onun içindeki sayım + kuyruktaki sayım (recurse her iki tarafa).

Örnekler:


```

(x '(a b c))           → 3
(x '(a #f b))          → 2    ; #f sayılmıyor
(x '((a b) c (d (e f)))) → 6    ; derinlemesine
(x '((a #f) b (c #f d))) → 4    ; nested + #f filter
(x '())                → 0

```

Pratik kullanım: Bool flag'lerle elenmiş, nested yapıda "anamlı" atom sayma.

Programming Exercises (22 soru)

Not: PE 13-22 daha uzun (sort, permutation, quicksort) — kapsamlı çözümler veriyorum. PE 21 ve 22 "bir önceki 19 alıştırmayı F#/ML'de yaz" diyor; bizim odak Scheme olduğu için onları geçiyorum.

PE 1. Write a Scheme function that computes the volume of a sphere, given its radius.

TR: Yarıçapı verildiğinde kürenin hacmini hesaplayan Scheme fonksiyonu.

Formül: $V = (4/3) \times \pi \times r^3$

```

(define pi 3.14159265358979)

(define (sphere-volume r)
  (* (/ 4 3) pi (* r r r)))

(sphere-volume 5)    ; → 523.598...

```

Not: Scheme'de `(/ 4 3)` rasyonel olarak $4/3$ dönebilir (implementation'a bağlı). Float kullanmak için `(/ 4.0 3.0)`.

PE 2. Write a Scheme function that computes the real roots of a given quadratic equation. If the roots are complex, the function must display a message indicating that. This function must use an IF function.

TR: Kuadratik denklemin reel köklerini hesapla. Kompleks ise mesaj göster. IF kullan.

$$ax^2 + bx + c = 0 \rightarrow x = (-b \pm \sqrt{b^2 - 4ac}) / 2a$$

```
(define (quadratic-roots a b c)
  (let ((discriminant (- (* b b) (* 4 a c))))
    (if (< discriminant 0)
        (display "Kompleks kökler – reel kök yok.")
        (let ((sqrt-d (sqrt discriminant)))
          (list (/ (+ (- b) sqrt-d) (* 2 a))
                (/ (- (- b) sqrt-d) (* 2 a)))))))

(quadratic-roots 1 -3 2)      ; → (2 1)          ; x² - 3x + 2
(quadratic-roots 1 0 1)      ; "Kompleks kökler ..."
(quadratic-roots 1 -2 1)      ; → (1 1)          ; (x-1)² = 0
```

PE 3. Repeat Programming Exercise 2 using a COND function, rather than an IF function.

TR: PE 2'yi COND ile yaz.

```
(define (quadratic-roots a b c)
  (let ((discriminant (- (* b b) (* 4 a c))))
    (cond
      ((< discriminant 0)
       (display "Kompleks kökler – reel kök yok.))
      (= discriminant 0)
      (list (/ (- b) (* 2 a))))          ; tek kök
    (else
     (let ((sqrt-d (sqrt discriminant)))
       (list (/ (+ (- b) sqrt-d) (* 2 a))
             (/ (- (- b) sqrt-d) (* 2 a))))))))
```

COND ile üç ayrı durumu (kompleks, tek kök, iki kök) ayırmak daha şık.

PE 4. Write a Scheme function that takes two numeric parameters, A and B, and returns A raised to the B power.

TR: A üzeri B (integer üs).

Built-in `expt` var (`(expt 2 10) → 1024`), ama recursion ile yazılacaksa:

```

(define (power a b)
  (cond
    ((= b 0) 1)
    ((= b 1) a)
    ((even? b) (let ((half (power a (/ b 2))))
                  (* half half))) ; A^(2k) = (A^k)^2
    (else (* a (power a (- b 1))))) ; A^n = A × A^(n-1)

(power 2 10) ; → 1024
(power 3 4) ; → 81

```

Negatif üs için: ekstra $(< b 0) \rightarrow (/ 1 (\text{power } a (- b)))$ eklenebilir.

PE 5. Write a Scheme function that returns the number of zeros in a given simple list of numbers.

TR: Simple liste içindeki sıfır sayısı.

```

(define (count-zeros lst)
  (cond
    ((null? lst) 0)
    ((= (car lst) 0) (+ 1 (count-zeros (cdr lst))))
    (else (count-zeros (cdr lst)))))

(count-zeros '(0 1 0 2 0 3)) ; → 3
(count-zeros '(1 2 3)) ; → 0
(count-zeros '()) ; → 0

```

PE 6. Write a Scheme function that takes a simple list of numbers as a parameter and returns a list with the largest and smallest numbers in the input list.

TR: Simple listeden en büyük ve en küçük sayıyı bir liste olarak döndür.

```

(define (min-max lst)
  (define (helper lst lo hi)
    (cond
      ((null? lst) (list lo hi))
      ((< (car lst) lo) (helper (cdr lst) (car lst) hi))
      ((> (car lst) hi) (helper (cdr lst) lo (car lst)))
      (else (helper (cdr lst) lo hi))))
  (if (null? lst)
      '() ; boş listenin min/max yok
      (helper (cdr lst) (car lst) (car lst))))

(min-max '(3 1 4 1 5 9 2 6 5)) ; → (1 9)
(min-max '(42)) ; → (42 42)

```

Tail-recursive helper kullandık; tek geçişte min ve max'i birden bulur.

PE 7. Write a Scheme function that takes a list and an atom as parameters and returns a list identical to its parameter list except with all top-level instances of the given atom deleted.

TR: *Top-level'da verilen atomun tüm geçişlerini sil.*

```

(define (remove-atom a lst)
  (cond
    ((null? lst) '())
    ((equal? (car lst) a) (remove-atom a (cdr lst)))
    (else (cons (car lst) (remove-atom a (cdr lst))))))

(remove-atom 'b '(a b c b d b)) ; → (a c d)
(remove-atom 3 '(1 2 3 4 3)) ; → (1 2 4)
(remove-atom 'a '((a b) a (c a))) ; → ((a b) (c a)) ; nested 'a'lar kalıyor (top-level değil)

```

"Top-level" → sadece doğrudan elemanları sil; iç listelerdeki atomları silme.

PE 8. Write a Scheme function that takes a list as a parameter and returns a list identical to the parameter except the last element has been deleted.

TR: Son elemanı silinmiş listeyi döndür.

```
(define (remove-last lst)
  (cond
    ((null? lst) '())
    ((null? (cdr lst)) '()) ; tek eleman kaldı → sil
    (else (cons (car lst) (remove-last (cdr lst))))))

(remove-last '(a b c d)) ; → (a b c)
(remove-last '(x))      ; → ()
(remove-last '())       ; → ()
```

PE 9. Repeat Programming Exercise 7, except that the atom can be either an atom or a list.

TR: PE 7 ama "atom" yerine atom VEYA liste olabilir; o öğeyi sil.

```
(define (remove-elem e lst)
  (cond
    ((null? lst) '())
    ((equal? (car lst) e) (remove-elem e (cdr lst)))
    (else (cons (car lst) (remove-elem e (cdr lst))))))

(remove-elem '(a b) '((a b) c (a b) d)) ; → (c d)
(remove-elem 'x '(x y x z))             ; → (y z)
```

PE 7 ile neredeyse aynı; tek fark `equal?` kullanmak (atom için `eq?` yeterliydi, liste karşılaştırması için `equal?` şart).

PE 10. Write a Scheme function that takes two atoms and a list as parameters and returns a list identical to the parameter list except all occurrences of the first given atom in the list are replaced with

the second given atom, no matter how deeply the first atom is nested.

TR: *Listede (nested olabilir) ilk atomun tüm geçişlerini ikinci atomla değiştir.*

```
(define (replace-deep old new lst)
  (cond
    ((null? lst) '())
    ((list? (car lst))
     (cons (replace-deep old new (car lst))
           (replace-deep old new (cdr lst))))
    ((equal? (car lst) old)
     (cons new (replace-deep old new (cdr lst))))
    (else
     (cons (car lst) (replace-deep old new (cdr lst))))))

(replace-deep 'a 'X '(a b (a c (a d)) a))    ; → (X b (X c (X d)) X)
```

Anahtar: (list? (car lst)) ise içeri recurse — derinlik kontrolü buradan geliyor.

PE 11. Write a Scheme function that returns the reverse of its simple list parameter.

TR: *Simple listeyi ters çevir.*

İki versiyon — naive ve tail-recursive:

; Naive – $O(n^2)$ çünkü append her seferinde tüm sol yarıyı geziyor

```
(define (reverse-naive lst)
  (cond
    ((null? lst) '())
    (else (append (reverse-naive (cdr lst))
                  (list (car lst))))))
```

; Tail-recursive helper ile – $O(n)$

```
(define (reverse-fast lst) (reverse-helper lst '()))
(define (reverse-helper lst acc)
  (cond
    ((null? lst) acc)
    (else (reverse-helper (cdr lst) (cons (car lst) acc)))))
```

```
(reverse-fast '(a b c d e))    ; → (e d c b a)
```

Tail-recursive versiyon: her elemanı sırayla acc'in başına ekler, sonunda acc ters sırada birikmiş olur.

PE 12. Write a Scheme predicate function that tests for the structural equality of two given lists. Two lists are structurally equal if they have the same list structure, although their atoms may be different.

TR: İki listenin yapısal eşitliği — atomlar farklı olabilir ama yapı (nesting şekli ve uzunlukları) aynı.

```
(define (struct-equal? l1 l2)
  (cond
    ((and (null? l1) (null? l2)) #t)
    ((or (null? l1) (null? l2)) #f) ; biri boş diğeri değilse no
    ((and (list? (car l1)) (list? (car l2)))
     (and (struct-equal? (car l1) (car l2))
           (struct-equal? (cdr l1) (cdr l2))))
    ((or (list? (car l1)) (list? (car l2))) #f) ; biri liste diğeri atom
    (else (struct-equal? (cdr l1) (cdr l2)))) ; iki atom – aynı yapıda say

(struct-equal? '(a b (c d) e) '(1 2 (3 4) 5)) ; #t
(struct-equal? '(a b) '(a (b)))                ; #f
(struct-equal? '(a (b)) '(c (d)))                ; #t
```

PE 13. Write a Scheme function that returns the union of two simple list parameters that represent sets.

TR: İki simple liste'nin (set olarak) birleşimi.

```
(define (member? e lst)
  (cond
    ((null? lst) #f)
    ((equal? (car lst) e) #t)
    (else (member? e (cdr lst)))))

(define (union l1 l2)
  (cond
    ((null? l1) l2)
    ((member? (car l1) l2) (union (cdr l1) l2)) ; zaten l2'de
    (else (cons (car l1) (union (cdr l1) l2)))))

(union '(a b c) '(b c d))      ; → (a b c d)
(union '(1 2 3) '(4 5 6))     ; → (1 2 3 4 5 6)
(union '() '(a b))            ; → (a b)
```

PE 14. Write a Scheme function with two parameters, an atom and a list, that returns a list identical to the parameter list except with all occurrences, no matter how deep, of the given atom deleted. The returned list cannot contain anything in place of the deleted atoms.

TR: Atom listede ne kadar derin geçerse geçsin tümünü sil; yerlerine bir şey koyma.


```

(define (remove-deep a lst)
  (cond
    ((null? lst) '())
    ((list? (car lst))
     (cons (remove-deep a (car lst))
           (remove-deep a (cdr lst))))
    ((equal? (car lst) a)
     (remove-deep a (cdr lst))) ; sil, yerine bir şey koyma
    (else
     (cons (car lst) (remove-deep a (cdr lst))))))

(remove-deep 'x '(a x b (c x d (x e)) x f)) ; → (a b (c d (e)) f)

```

PE 10 ile karşılaştır: orada **replace**, burada **remove**. Replace'de eleman `new` 'ye dönüşüyor; remove'da kaybediyor.

PE 15. Write a Scheme function that takes a list as a parameter and returns a list identical to the parameter list except with the second top-level element removed. If the given list does not have two elements, the function should return ().

TR: *İkinci elemanı silinmiş liste. Liste 2 elemandan az ise () döndür.*

```

(define (remove-second lst)
  (cond
    ((null? lst) '())
    ((null? (cdr lst)) '()) ; tek eleman → ()
    (else (cons (car lst) (cddr lst)))) ; head + drop(2)

(remove-second '(a b c d)) ; → (a c d)
(remove-second '(x y)) ; → (x) ; wait – bunu yeniden okuyalım
(remove-second '(x)) ; → ()
(remove-second '()) ; → ()

```

Tekrar oku — $(x\ y)$ durumu: 2 eleman var, ikincisi silinince (x) → bu mantıklı. Spec'in "less than 2 → ()" yanlış uyguladık aslında. Tekrar:

```
(define (remove-second lst)
  (cond
    ((null? lst) '())
    ((null? (cdr lst)) '()) ; tek eleman, return ()
    (else (cons (car lst) (cddr lst))))) ; iki+ eleman, ikinciye sil
```

$(x\ y) : \text{lst } 2 \text{ eleman} \rightarrow (\text{cons } 'x\ '()) \rightarrow (x) . \checkmark$

PE 16. Write a Scheme function that takes a simple list of numbers as its parameter and returns a list identical to the parameter list except with the numbers in ascending order.

TR: Sayı listesini küçükten büyüğe sırala.

Insertion sort yaklaşımı:

```
(define (insert x sorted)
  (cond
    ((null? sorted) (list x))
    ((<= x (car sorted)) (cons x sorted))
    (else (cons (car sorted) (insert x (cdr sorted))))))

(define (sort-asc lst)
  (cond
    ((null? lst) '())
    (else (insert (car lst) (sort-asc (cdr lst)))))

(sort-asc '(3 1 4 1 5 9 2 6 5)) ; → (1 1 2 3 4 5 5 6 9)
```

Daha verimli (quicksort) için PE 19'a bak.

PE 17. Write a Scheme function that takes a simple list of numbers as its parameter and returns the largest and smallest numbers in the list.

TR: PE 6 ile aynı — min/max liste döndür.

PE 6'daki çözüm tam karşılığıdır. Tekrarlamıyorum.

PE 18. Write a Scheme function that takes a simple list as its parameter and returns a list of all permutations of the given list.

TR: *Bir listenin tüm permütasyonlarını döndür.*

```
(define (insert-everywhere x lst)
  ; x'i lst'nin her pozisyonuna yerleştir, sonuçların listesini döndür
  (cond
    ((null? lst) (list (list x)))
    (else
     (cons (cons x lst)
           (map (lambda (p) (cons (car lst) p))
                (insert-everywhere x (cdr lst)))))))

(define (permutations lst)
  (cond
    ((null? lst) '(())) ; boş listenin permütasyonu boş liste'dir
    (else
     (apply append
             (map (lambda (p) (insert-everywhere (car lst) p))
                  (permutations (cdr lst)))))))

(permutations '(a b c))
; → ((a b c) (b a c) (b c a) (a c b) (c a b) (c b a))
```

n eleman için $n!$ permütasyon. $O(n!)$ zaman ve bellek.

PE 19. Write the quicksort algorithm in Scheme.

TR: *Scheme'de quicksort.*

```

(define (filter pred lst)
  (cond
    ((null? lst) '())
    ((pred (car lst)) (cons (car lst) (filter pred (cdr lst))))
    (else (filter pred (cdr lst)))))

(define (quicksort lst)
  (cond
    ((null? lst) '())
    ((null? (cdr lst)) lst) ; tek eleman
    (else
     (let ((pivot (car lst))
           (rest (cdr lst)))
       (append
        (quicksort (filter (lambda (x) (< x pivot)) rest))
        (list pivot)
        (quicksort (filter (lambda (x) (>= x pivot)) rest)))))))

(quicksort '(3 1 4 1 5 9 2 6 5 3 5)) ; → (1 1 2 3 3 4 5 5 5 6 9)

```

Klasik functional quicksort. Average $O(n \log n)$, worst $O(n^2)$ (already-sorted input).

PE 20. Rewrite the following Scheme function as a tail-recursive function:

```

(DEFINE (doit n)
  (IF (= n 0)
      0
      (+ n (doit (- n 1)))))

```

TR: *Bu fonksiyonu tail-recursive hâle getir.*

Orijinal: $\text{doit}(n) = n + (n-1) + (n-2) + \dots + 1 + 0$ — 0'dan n'e toplam.

NOT tail recursive — çağrıdan sonra $(+ n \dots)$ var.

Accumulator ile tail-rec çözüm:

```

(define (doit n) (doit-helper n 0))

(define (doit-helper n acc)
  (if (= n 0)
      acc
      (doit-helper (- n 1) (+ acc n))))

(doit 5)      ; → 15
(doit 100)    ; → 5050

```

Her iki recursion noktasında çağrı SON işlem; $(+ \text{acc } n)$ çağrıdan önce hesaplandı. Scheme TCO ile bunu sabit-stack loop'a derler.

PE 21–22. Write any of the first 19 Programming Exercises in F# / ML.

TR: *PE 1-19'dan herhangi birini F#/ML'de yaz.*

(F# örneği — PE 5 sıfır sayma):

```

let rec countZeros lst =
  match lst with
  | [] -> 0
  | 0 :: tail -> 1 + countZeros tail
  | _ :: tail -> countZeros tail

```

(ML örneği — PE 7 atom silme):

```

fun removeAtom a [] = []
  | removeAtom a (x::xs) =
    if x = a then removeAtom a xs
    else x :: removeAtom a xs

```

F#/ML'in pattern matching syntaxı Scheme'in COND'undan daha şık — case label'lar pattern olarak doğrudan.

Sınav odaklı bizim için bunlar düşük öncelikli; Scheme versiyonlarını sağlam öğrenmek yeterli.

Chapter 16 — Logic Programming Languages

Review Questions (25 soru)

1. What are the three primary uses of symbolic logic in formal logic?

TR: Sembolik mantığın formal logic'teki üç temel kullanımı?

1. **Önergeleri kesin biçimde ifade etmek** (formal expression of propositions).
2. **Önergeler arası ilişkileri kesin tanımlamak** (expressing relationships among propositions).
3. **Yeni önergelerin önceki önergelerden çıkarılma yöntemleri** (methods of inference — yeni doğruları kanıtlama).

Prolog #3'ü makinaya dökme girişimidir.

2. What are the two parts of a compound term?

TR: Compound term'in iki parçası nedir?

1. **Functor** — fonksiyon/yüklem ismi (atom).
2. **Argument list** — parantezli, virgülle ayrılmış argümanlar.

Örnek: `parent(tom, alice)`. Functor = `parent`, args = `[tom, alice]`. Arity 2 (`parent/2`).

3. What are the two modes in which a proposition can be stated?

TR: Bir önerme hangi iki modda ifade edilebilir?

1. **Hipotez (hypothesis)**: Doğru olarak iddia edilen veya kanıtlanması istenen.
2. **Teorem (theorem)**: Diğer önergelerden mantıksal olarak çıkarılan.

Prolog'da: facts ve rules birer hipotezdir (database'de doğru sayılır); query ise bir teorem kanıtlama girişimidir.

4. What is the general form of a proposition in clausal form?

TR: *Clausal form'da bir önermenin genel biçimi?*

$$B_1 \vee B_2 \vee \dots \vee B_n \leftarrow A_1 \wedge A_2 \wedge \dots \wedge A_m$$

(B'ler "sonuçlar" — birinin doğru olması yeter; A'lar "önkoşullar" — hepsinin doğru olması gerek.)

Türkçe okuma: "Tüm A_i'ler doğruysa, en az bir B_j doğrudur."

5. What are antecedents? Consequences?

TR: *Antecedent (önkoşul) ve consequent (sonuç) nedir?*

- **Antecedent:** Bir koşullu önermenin koşul (önkoşul) tarafı. Yukarıdaki formdaki A'lar.
- **Consequent:** Sonuç tarafı. B'ler.

"If A then B" — A antecedent, B consequent.

Prolog rule'da: head :- body. — head consequent, body antecedent.

6. Give general (not rigorous) definitions of resolution and unification.

TR: *Resolution ve unification'ın gevşek tanımları.*

- **Resolution:** İki clause'u, ortak bir literal (biri pozitif diğeri negatif) üzerinden eşleştirip yeni bir clause türetme inference kuralı. Prolog'un motor mantığı SLD-resolution'dır.
- **Unification:** İki terimi (ya da daha fazlasını) eşit yapacak bir değişken-atama (substitution) bulma. Mümkünse en genel substitution (MGU = most general unifier).

İkisi birlikte: Resolution + Unification = Prolog'un theorem proving motoru.

7. What are the forms of Horn clauses?

TR: *Horn clause'un formları nelerdir?*

Horn clause = en fazla **bir** pozitif literal içeren clausal form.

Üç alt form:

1. **Headed (definite) clause:** Tam bir pozitif literal var.

$$B \leftarrow A_1 \wedge \dots \wedge A_n$$

(Prolog'da: $B :- A_1, A_2, \dots, A_n.$ — yani **rule**.)

2. **Headless clause:** Hiç pozitif literal yok.

$$\leftarrow A_1 \wedge \dots \wedge A_n$$

(Prolog'da: $?- A_1, A_2, \dots, A_n.$ — yani **goal/query**.)

3. **Fact (özel headed):** Sadece head var, body yok.

$$B \leftarrow \text{veya } B.$$

(Prolog'da: `parent(tom, alice).`)

8. What is the basic concept of declarative semantics?

TR: *Declarative semantik'in temel kavramı?*

Programın anlamı, "ne hesaplandığı" üzerinden tanımlanır — "nasıl hesaplandığı" değil. Programcı problemin tanımını verir; sistem çözmeyi düşünür.

İmperative'de: program = bir komut dizisi (nasıl yap).

Declarative'de: program = problemin tanımı (ne istiyorum).

Prolog declarative — facts ve rules verirsin; engine $?-$ ile sorgulanan teoremi kanıtlar. SQL de declarative (`SELECT ... WHERE ...`).

9. What does it mean for a language to be nonprocedural?

TR: *Nonprocedural dil ne demek?*

Declarative ile yakın anlamı — "prosedürel olmayan" — yani algoritma adımlarını kullanıcı yazmaz, problem ifadesini yazar. Çözüm motorunun sorumluluğu.

Prolog tam saf nonprocedural değildir — clause sırası, cut, side effect'leri ile prosedürel bir okuma da edinilir. "Mostly declarative + a procedural escape hatch."

10. What are the three forms of a Prolog term?

TR: *Prolog term'in üç formu?*

1. **Constants (atoms ve numbers):** tom , alice , 42 , 3.14 , [] .
2. **Variables:** Büyük harfle başlayan veya _ ile: X , Result , _Temp .
3. **Compound terms (structures):** functor(arg1, arg2, ...) . Örn parent(tom, alice) , [a, b, c] .

Listeler aslında özel compound term'ler: [H|T] $\equiv$ '.'(H, T) .

11. What is an uninstantiated variable?

TR: *Uninstantiated (henüz değer almamış) değişken nedir?*

Henüz bir değere bind edilmemiş — "boş" değişken. Prolog'da unification ile birine bağlanır:

```
?- X = 5.           % X şimdi 5'e instantiated
?- parent(tom, X).  % engine X'i alice ya da bob'a instantiate eder
```

Uninstantiated variable, arithmetic'te (is) sağ tarafta varsa **hata** verir (insufficient instantiation).

12. What are the syntactic forms and usage of fact and rule statements in Prolog?

TR: *Prolog'da fact ve rule deyimlerinin söz dizimi ve kullanımı.*

Fact: Koşulsuz doğru olan bir önerme.

```
parent(tom, alice).
loves(alice, bob).
```

Rule: Bir koşula bağlı doğruluk.

```
% head :- body.
mother(M, C) :- parent(M, C), female(M).
grandparent(G, C) :- parent(G, P), parent(P, C).
```

, AND, ; OR. :- "if". Sonu nokta . ile biter.

Kullanım: Fact ve rule'lar Prolog database'ini (knowledge base) oluşturur. Query'ler bu database'e karşı çalıştırılır.

13. What is a conjunction?

TR: *Conjunction nedir?*

İki ya da daha fazla terim/literal'i AND ile birleştirme. Prolog'da `,` ile yazılır.

```
?- parent(tom, X), parent(X, alice).  
% "tom X'in parent'ı VE X alice'in parent'ı – yani tom alice'in grandparent'ı."
```

Tüm konjunkt'ler birlikte true olmazsa, conjunction false.

14. Explain the two approaches to matching goals to facts in a database.

TR: *Database'de goal'ları facts'a eşlemenin iki yaklaşımı.*

1. **Forward chaining (data-driven):** Fact'lerden başla; rule'ları uygulayarak yeni fact'ler türet; hedef fact'e ulaşıncaya dur.
 - Kullanım: expert system'ler, "ne kanıtlanabilir?" sorusu.
 - Sorun: Çok fact varsa explosion (geniş hipotez uzayı).
2. **Backward chaining (goal-driven):** Goal'dan başla; goal'u sağlayacak rule'u bul; rule'un body'sindeki alt-goal'lar için aynı süreci tekrarla; en sonunda fact'lere ulaş.
 - Prolog **backward chaining** kullanır.
 - Pratik: "amacın ne kanıtlanmasını?" sorusunu doğrudan ele alır.

15. Explain the difference between a depth-first and a breadth-first search when discussing how multiple goals are satisfied.

TR: *Multiple goal satisfaction'da depth-first ve breadth-first farkı.*

- **Depth-first:** İlk alt-goal'u tamamen çöz, sonra ikinci alt-goal'a geç. Çıkmaz sokağa girersen geri dön (backtrack). Prolog bunu kullanır.
- **Breadth-first:** Tüm alt-goal'ları paralel olarak bir seviye ilerlet. Hafıza ağırlığı yüksek.

Trade-off:

- Depth-first: az hafıza, hızlı (genelde), ama sonsuz tree dallarına saplanabilir → completeness yok.
- Breadth-first: complete (bir çözüm varsa bulur), ama exponential hafıza.

Prolog'un depth-first seçimi pratik bir uzlaşma — programcı goal/clause sırasını ayarlayarak çıkmazları yönetebilir.

16. Explain how backtracking works in Prolog.

TR: *Prolog'da backtracking nasıl çalışır?*

1. Bir goal'ı çözmeye çalış: ilk uygun rule/fact ile unify etmeyi dene.
2. Başarılıysa: o rule'un body'sinde sıradaki alt-goal'a geç.
3. Başarısızsa (unify edemedi veya alt-goal fail): **geri dön** — son **choice point**'e (rule seçim noktasına) git.
4. O noktada başka bir rule'u dene.
5. Tüm rule'lar tükenirse bir üst seviyede backtrack.
6. Bütün ağaç tükenirse final fail.

Backtracking, **non-determinism**'in implementation'ıdır — "birden fazla seçim varsa, ilki çözmezse diğerini dene." Kullanıcı ; ile alternatif cevapları manuel tetikleyebilir.

17. Explain what is wrong with the Prolog statement $K \text{ is } K + 1$.

TR: *$K \text{ is } K + 1$ Prolog'da niye yanlış?*

İki ihtimal:

(a) K uninstantiated: Sağ taraf $K + 1$ — K henüz bilinmiyor $\rightarrow$ is aritmetik için sağ tarafın bilinmesini ister $\rightarrow$ **insufficient instantiation error**.

(b) K instantiated (örneğin $K = 5$): $5 \text{ is } 5 + 1 \rightarrow 5 \text{ is } 6$ — bunu unify etmeye çalışır $\rightarrow$ **fail** ($5 \neq 6$).

Hiçbir senaryoda imperative $K = K + 1$ (yani K'yi 1 artır) anlamına gelmez. **Prolog'da variable bind edildi mi değişmez** — single assignment. Counter mantığı için recursive call'a yeni bir parametre ile gitmek gerek.

18. What are the two ways a Prolog programmer can control the order of pattern matching during resolution?

TR: *Prolog programcısı pattern matching sırasını nasıl kontrol eder?*

1. **Clause ordering:** Database'de fact/rule'ların sırasını değiştirerek. Prolog yukarıdan aşağı denediği için, daha sık çözen veya daha temel olan clause'u öne almak iyi pratiktir.
2. **Cut (!) operatörü:** Backtracking'i kısıtlar. ! geçildikten sonra:
 - O predicate için kalan clause'lar denenmez.
 - Aynı rule'un body'sinde solda kalan goal'lar revize edilmez.

Cut, "şu noktaya kadar yaptıklarımı kabul et, geri dönüp başka seçim arama" demektir. Verimliliği artırır ama declarative anlamı zayıflatır.

19. Explain the generate-and-test programming strategy in Prolog.

TR: *Prolog'da generate-and-test stratejisi.*

İki adımlı problem çözümü:

1. **Generate:** Bir aday (kısmi) çözüm türet (non-deterministic).
2. **Test:** Aday gerçekten geçerli mi diye test et. Başarısız olursa backtrack → yeni aday generate et.

Prolog'un backtracking + unification'ı bu kalıbı doğal kılar:

```
% Bir sayı seç ve "asal ve <100" mı diye test et
solve(X) :- between(1, 100, X), prime(X).
```

between/3 ardı ardına 1,2,3,... üretir; prime(X) test eder; fail ederse bir sonraki aday.

Kombinatorial arama, constraint satisfaction, planning gibi alanlarda ana stratejidir. Performans iyi olabilmesi için **fail-early** prensibine dikkat — testi mümkün olduğunca öne çek.

20. Explain the closed-world assumption used by Prolog. Why is this a limitation?

TR: *Closed-world assumption nedir, niye sınırlama?*

Closed-world assumption (CWA): Database'de yer almayan bir önerme **yanlış** sayılır. Bilgi tamdır varsayımı.

```
parent(tom, alice).  
% Hiç başka parent fact'i yok.  
  
?- parent(tom, bob).  
% false → Prolog "bilmiyorum" demez, "yanlış" der.
```

Sınırlamalar:

1. **Gerçek dünyaya uymaz:** Bir şey database'de yok ≠ doğru değil. Bilgisizlik ile yanlışlık karışır.
2. **\+ Goal (negation as failure):** Negasyon "Goal'u kanıtlayamadım" anlamında, gerçek negation değil.
 \+ parent(tom, bob) → true; ama bob gerçekten tom'un çocuğu olabilir, sadece DB'de yok.
3. **Updates problemli:** Yeni fact eklendiğinde, önceki "false" sonuçlar invalidate olur. Monotonicity yoktur — eklemeye azalmaya gidebilir.

Modern logic system'ler bunun için "three-valued logic" (true/false/unknown) veya **autoepistemic** mantıklar geliştirdi.

21. Explain the negation problem with Prolog. Why is this a limitation?

TR: *Prolog negation probleminin niye bir sınırlama?*

Soru 20'nin devamı. Prolog'un \+ Goal (negation as failure) Goal 'ü kanıtlayamadıysa true sayar.

Sınırlama 1 — yanlış sonuçlar:

```
likes(mary, ice_cream).  
?- \+ likes(john, ice_cream).  
% true → ama gerçekte bilmiyoruz!
```

Sınırlama 2 — uninstantiated değişkenlerle yanlış davranış:

```
?- \+ likes(X, ice_cream).  
% likes(mary, ice_cream) tüm X için fail değil → false döner  
% ama "X'ler var ki ice_cream sevmez" doğru olabilir
```

Sınırlama 3 — non-monotonic: Yeni fact eklemek önceden "true" olan \+ sonuçlarını "false" yapabilir. Klasik mantıkta yeni doğru eklemek önceki doğrulara eklemeye yapar; Prolog'da öyle değil.

Çözüm girişimleri: "negation as inconsistency" (only assert "not" when consistent), three-valued logic, ASP (Answer Set Programming) gibi paradigmlar.

22. Explain the connection between automatic theorem proving and Prolog's inferencing process.

TR: *Automatic theorem proving ve Prolog inferencing bağı.*

Prolog **resolution-based theorem prover**'dir:

1. Database = aksiyom kümesi (verili doğrular).
2. Query = kanıtlanmak istenen teorem (`?- goal.`).
3. Engine: query'i yürütme (refutation) yoluyla kanıtlamaya çalışır — query'in negasyonunu aksiyomlara ekler, çelişki (boş clause) arar.
4. Bulursa: original query doğru (theorem) demektir → "true" ve değişken atamaları.
5. Bulamazsa: kanıtlanamadı → "false" (CWA altında).

SLD-resolution (Selective Linear resolution for Definite clauses) Prolog'un kullandığı strateji. Robinson'ın 1965 resolution principle'inin özel hâli.

Pratikte: Prolog tam bir theorem prover değil — Horn clause'lara sınırlı, depth-first → eksik. Tam theorem prover'lar (HOL, Coq, Isabelle) daha güçlü ama interactive.

23. Explain the difference between procedural and nonprocedural languages.

TR: *Procedural vs nonprocedural dil farkı. (kısa)*

- **Procedural:** "Nasıl yap" — adım adım komut listesi. C, Java, Python.
- **Nonprocedural (declarative):** "Ne yap" — sonucun ne olacağını ifade et, sisteme nasıl ulaşacağını bırak. Prolog, SQL, Haskell (saf), domain-specific declarative diller.

24. Explain why Prolog systems must do backtracking.

TR: *Prolog niye backtracking yapmak zorunda?*

Çünkü Prolog'un mantığı **non-deterministic**: bir goal'ı sağlamak için **birden fazla rule veya fact** uygun olabilir. Engine bir tane seçer, sonra sonraki goal'a geçer. Eğer sonraki adımlarda bu seçim çıkmaza yol açarsa, geri dönüp **başka seçimi** denemek gerek → backtracking.

Backtracking olmazsa Prolog yalnız "ilk olası çözümde takıldıysa fail" der; çözümlü dalları kaçıırır.

Multiple solutions için (; ile) de gerek — kullanıcı "başka cevap var mı?" sorusunu sorduğunda, engine son choice point'e döner ve devam eder.

25. What is the relationship between resolution and unification in Prolog?

TR: *Prolog'da resolution ve unification ilişkisi.*

Resolution iki clause'u birleştirip yeni bir clause türetir. Bu birleşme için iki literal'in **eşit** olması gerek — ama literal'ler değişken içerir, doğrudan eşit olmayabilirler.

Unification, bu literal'leri eşitleyecek değişken atamalarını bulur. Resolution adımı:

1. Bir literal seç (örn. soldaki ilk goal).
2. Database'den bir clause seç, ki onun head'i bu literal'le **unify edilebilsin**.
3. Unification'dan gelen substitution'u uygula.
4. Goal'u clause'un body'siyle değiştir.
5. Tekrarla.

Yani: **Resolution = inference framework, unification = pattern matching mekanizması**. İkisi olmadan biri çalışmaz.

% Resolution adımı örneği:

```
parent(tom, alice).           % database fact
?- parent(tom, X).            % query
```

% Unify parent(tom, alice) $\triangleright$ parent(tom, X) $\rightarrow$ X = alice

% Resolution: query çözüldü, X = alice

Problem Set (10 soru)

1. Compare the concept of data typing in C# with that of Prolog.

TR: C# ve Prolog'da data typing karşılaştırması.

C# — statically typed:

- Her değişkenin tipi compile-time'da belli.
- Tip kontrolü compile aşamasında yapılır → hatalar erken yakalanır.
- Tip sistemi: primitive (int, double, char, bool), class, interface, struct, enum, generic.
- Implicit conversion kuralları var (widening); explicit cast narrowing için.
- Type inference (var) destekli; ama tip yine de derleme-zamanı belirli.

Prolog — dynamically typed (tipsiz benzeri):

- Term'lerin tipleri çalışırken belli (atom mı, sayı mı, liste mi?).
- Tip kontrolü için atom/1 , number/1 , is_list/1 gibi predicate'ler var, ama programcının bilinçli kullanması gerek.
- Tip uyumsuzluğu çoğunlukla **unification failure** olarak ortaya çıkar (örn. parent(X, Y) ile parent(5) unify olmaz — arity farklı).
- Aritmetik için tip kontrolü is operatöründe yapılır — sağ taraf sayısal değilse error.

Karşılaştırma:

- C# güvenli ama esnek değil; Prolog esnek ama sessiz bug imkânı yüksek.
- C# generic'leri ile reusable kod; Prolog'un polymorphism'i doğal (tüm değişkenler "any term" kabul eder).
- Hata bulma: C# compile-time; Prolog runtime — debugging farklı.

2. Describe how a multiple-processor machine could be used to implement resolution. Could Prolog, as currently defined, use this method?

TR: Çoklu işlemcili makine resolution'ı nasıl gerçekleyebilir? Şu anki Prolog bunu kullanabilir mi?

Paralel resolution stratejileri:

1. **OR-parallelism:** Birden fazla rule alternatifi paralel olarak denenir. Aynı goal için birden fazla rule varsa, her processor bir clause'u dener; çözüm bulan kazanır.

2. **AND-parallelism:** Bir rule'un body'sindeki bağımsız alt-goal'lar paralel çözülür. Veri bağımlılığı (bir alt-goal başkasının instantiation'una bağlıysa) sırayı zorlar.
3. **Independent AND-parallelism:** Bağımsız body conjunct'leri (paylaşılan değişkensiz) tam paralel.

Şu anki Prolog'la sorun:

- Standart Prolog **sıralı semantik** garanti eder — clause sırası, side effect sırası önemli. Cut bu sıralı semantiğe bağlı.
- Side effect'ler (assert, retract, write) paralel ortamda kaos.
- Backtracking'in geri dönüş yolu paralel'de karmaşık.

Bu nedenle özel paralel Prolog implementation'ları geliştirildi: Aurora, Andorra-I, &-Prolog. Bunlar pure mantıksal alt-küme + bazı extension'lar üzerinde çalışır. Tam standart Prolog'u doğru paralel-leştirmek hâlâ açık problem.

3. Write a Prolog description of your family tree (based only on facts), going back to your grandparents and including all descendants. Be sure to include all relationships.

TR: *Aile ağacını sadece fact'lerle Prolog'da yaz; büyükanneler/babalardan torunlara kadar.*

Örnek (3 nesil):

```
% Gender
male(ahmet).
male(mehmet).
male(ali).
male(cagri).
female(fatma).
female(ayse).
female(elif).
female(zeynep).

% Parent (P, C) → P, C'nin parent'ı
parent(ahmet, ali).
parent(fatma, ali).
parent(mehmet, ayse).
parent(elif, ayse).
parent(ali, cagri).
parent(ayse, cagri).
parent(ali, zeynep).
parent(ayse, zeynep).

% Marriage
married(ahmet, fatma).
married(mehmet, elif).
married(ali, ayse).
```

Bu sadece fact tabanı — rule'lar bir sonraki soruda. Tüm ilişkiler için ek fact'ler (great-grandparent vs.) eklenebilir ama o detay rule'larla çıkarılabilir → 4. soruda.

4. Write a set of rules for family relationships, including all relationships from grandparents through two generations. Now add these to the facts of Problem 3, and eliminate as many of the facts as you can.

TR: *Family ilişki rule'ları yaz; Problem 3'teki bir kısım fact'i rule'la türetilebilir hâle getir.*

```

% Türetilmiş ilişkiler
mother(M, C) :- parent(M, C), female(M).
father(F, C) :- parent(F, C), male(F).

child(C, P) :- parent(P, C).
son(S, P) :- child(S, P), male(S).
daughter(D, P) :- child(D, P), female(D).

sibling(X, Y) :- parent(P, X), parent(P, Y), X \= Y.
brother(X, Y) :- sibling(X, Y), male(X).
sister(X, Y) :- sibling(X, Y), female(X).

grandparent(GP, GC) :- parent(GP, P), parent(P, GC).
grandfather(GF, GC) :- grandparent(GF, GC), male(GF).
grandmother(GM, GC) :- grandparent(GM, GC), female(GM).

% Amca/teyze (parent'ın sibling'i)
uncle(U, N) :- parent(P, N), sibling(U, P), male(U).
aunt(A, N) :- parent(P, N), sibling(A, P), female(A).

% Cousin (parent'ın sibling'inin child'ı)
cousin(C1, C2) :- parent(P1, C1), parent(P2, C2),
                  sibling(P1, P2), C1 \= C2.

% Ancestor – recursive
ancestor(A, D) :- parent(A, D).
ancestor(A, D) :- parent(A, X), ancestor(X, D).

```

Eliminate edilen fact'ler: Problem 3'teki tüm `mother/2` , `father/2` , `grandparent/2` , `sibling/2` , `uncle/2` , vs. ilişkiler — bunlar artık rule'larla türetiliyor. Sadece `parent` , `male` , `female` atomic fact olarak kalıyor. Marriage ayrıca tutulmalı.

5. Write the following English conditional statements as Prolog-headed Horn clauses:

- If Fred is the father of Mike, then Fred is an ancestor of Mike.
- If Mike is the father of Joe and Mike is the father of Mary, then Mary is the sister of Joe.
- If Mike is the brother of Fred and Fred is the father of Mary, then Mike is the uncle of Mary.

TR: İngilizce koşullu ifadeleri Prolog headed Horn clause olarak yaz.

```

% (a)
ancestor(Fred, Mike) :- father(Fred, Mike).

% Daha genel: father olan ancestor olur (her ikili için)
ancestor(F, C) :- father(F, C).

% (b) Sister relation
sister(Mary, Joe) :- father(Mike, Joe), father(Mike, Mary), Mary \= Joe, female(Mary).

% Daha genel:
sister(S, B) :- father(F, B), father(F, S), S \= B, female(S).
% (Mother için de eklemek istersek):
sister(S, B) :- mother(M, B), mother(M, S), S \= B, female(S).

% (c) Uncle relation
uncle(Mike, Mary) :- brother(Mike, Fred), father(Fred, Mary).

% Daha genel:
uncle(U, N) :- brother(U, F), father(F, N).
% Mother tarafından amca/dayı için:
uncle(U, N) :- brother(U, M), mother(M, N).

```

Verilen önermeler **özel durumlar** — gerçek family tree için **genel** rule'lar yazılmalı (genelleştirilmiş versiyonlar). Soru özel olduğu için her ikisini de verdim.

6. Explain two ways in which the list-processing capabilities of Scheme and Prolog are similar.

TR: *Scheme ve Prolog'un liste işleme yeteneklerinin iki benzerliği.*

1. **Head/Tail decomposition** — Her ikisi de listenin head'ini ve tail'ini ayırtırmaya dayalı:
 - Scheme: (CAR lst) , (CDR lst) , (CONS head tail) .
 - Prolog: [H|T] pattern, [H|T] = [a,b,c] .
2. **Recursive iteration** — Her ikisinde de liste işleme tipik recursive: base case (empty list) + recursive case (head'i işle + tail'e recurse).

```

; Scheme – length
(define (len lst)
  (cond ((null? lst) 0)
        (else (+ 1 (len (cdr lst))))))

```

```
% Prolog – length
mylen([], 0).
mylen([_|T], N) :- mylen(T, N1), N is N1 + 1.
```

Yapısal olarak iki dil neredeyse aynı algoritma kalıbını kullanır.

7. In what way are the list-processing capabilities of Scheme and Prolog different?

TR: *Scheme ve Prolog'un liste işleme yeteneklerinin farkı.*

1. Unification vs explicit decomposition:

- Prolog'da $[H, S \mid T] = [a, b, c, d]$ pattern matching ile bir adımda multiple eleman çıkarır: $H=a, S=b, T=[c,d]$.
- Scheme'de aynı şey için `(car lst)`, `(cadr lst)`, `(cddr lst)` ayrı çağrılar.

2. Bidirectionality:

- Prolog rule'ları (örn. `append/3`) hem **forward** (concatenate) hem **backward** (split) çalışabilir:
`append([1,2], [3,4], X) → X=[1,2,3,4]`; `append(X, Y, [1,2,3,4]) → tüm split'leri üretir.`
- Scheme fonksiyonları tek yönlüdür — argümanlardan sonuca, ters yön yok.

3. First-class:

- Scheme listelerini fonksiyon argümanı ve dönüşü olarak kullanmak çok doğal.
- Prolog'da liste term'dir, fonksiyon değil — predicate'ler argüman olarak liste alır.

8. Write a comparison of Prolog with ML, including two similarities and two differences.

TR: *Prolog ve ML karşılaştırması — 2 benzerlik 2 fark.*

Benzerlikler:

1. **Pattern matching merkezli:** İkisi de fonksiyon/rule'ları pattern'lerle tanımlar.
2. **Recursive bağımlılık:** Loop yok; her ikisinde de recursion temel kontrol akışı.

Farklılıklar:

1. **Paradigma:** ML functional (fonksiyon = matematiksel mapping). Prolog logic-based (deklaratif önerme çözücü).

2. **Tip sistemi:** ML statik tip + type inference (Hindley-Milner). Prolog tipsiz (dynamic) — değişkenler herhangi bir term'e bind olabilir.
3. (Ekstra) **Backtracking:** Prolog'un kalbi. ML strict execution, geri dönüş yok.
4. (Ekstra) **Birden çok çözüm:** Prolog ; ile alternative cevaplar üretir. ML fonksiyonu tek değer döner.

9. From a book on Prolog, learn and write a description of an occur-check problem. Why does Prolog allow this problem to exist in its implementation?

TR: *Occur-check problem nedir? Prolog niye bu sorunu var ediyor?*

Occur-check problem:

Unification sırasında bir değişkenin **kendisine bağlı bir term**'le unify edilmesi sorgulanmalı:

?- $X = f(X).$

Mantıksal olarak imkânsız — X bir term'e eşit olmalı ama o term zaten X 'i içeriyor → sonsuz nested yapı. Standart unification algoritması (Robinson 1965) **occur-check** ile bunu yakalar: X 'i unify etmeden önce, sağ tarafta X 'in geçip geçmediğini kontrol et.

Prolog'un sorunu: Performans nedeniyle, çoğu Prolog implementation'ı **occur-check'i atlar**. Bu sayede unification $O(n)$ yerine $O(n^2)$ 'den kaçır (büyük yapıda büyük fark). Sonuç: yukarıdaki örnek implementation'da "true" döner ve X bir circular pointer'a bind olur — sonradan write_term denemesi infinite loop'a girer.

Niye Prolog izin veriyor:

1. **Performans:** Tipik programlarda bu sorun pratik olarak doğmaz; her unification'da check yapmak gereksiz overhead.
2. **Geleneksel davranış:** Erken Prolog sistemleri (1970'ler) check'siz yapıldı; geriye dönük uyumluluk korundu.
3. **ISO standart:** unify_with_occurs_check/2 ayrı predicate olarak verildi — kullanıcı isterse açıkça çağırır.

Pratik öneri: Critical kodda (özellikle değişken bir compound term'e bind edilirse) explicit occur-check kullan veya kod tasarımıyla durumdan kaçın.

10. Find a good source of information on Skolem normal form and write a brief but clear explanation of it.

TR: *Skolem normal form nedir?*

Skolem normal form (SNF): First-order logic formüllerinin "varolur ($\exists$) niceleyicisi taşımayan" prenex normal form'a dönüştürülmüş hâli.

Adımlar:

1. **Prenex normal form'a çevir** — tüm quantifier'ları formülün başına çek.

$$\forall x \exists y P(x, y) \wedge \exists z Q(z) \rightarrow \forall x \exists y \exists z (P(x, y) \wedge Q(z))$$

2. **Skolemization** — her $\exists$ quantifier'ı, dış $\forall$ 'lara bağlı bir **Skolem function** ile değiştir.

- Bağımlılık varsa $\rightarrow$ fonksiyon. $\exists y$ outer $\forall x$ 'e bağlı: $y \rightarrow f(x)$.
- Bağımlılık yoksa $\rightarrow$ Skolem constant. $\exists z$ bağımsız: $z \rightarrow c$.

Örnek dönüşüm:

$$\forall x \exists y \exists z (P(x, y) \wedge Q(z)) \rightarrow \forall x (P(x, f(x)) \wedge Q(c))$$

3. **Sonuç:** Sadece $\forall$ quantifier'ları (en başta) ve onların kapsadığı formül kaldı.

Niye gerek: Resolution-based theorem proving (Prolog dahil) **clausal form** ister; clausal form'a giden adımdan biri Skolem normal form'dur. Quantifier-free hâle gelmek inference için şart.

Pratik: SNF dönüşümü genel formülün doğruluğunu **korumaz** ama "satisfiability"sini korur — bir formül satisfiable ise SNF de satisfiable.

Programming Exercises (7 soru)

1. Using the structures `parent(X, Y)`, `male(X)`, and `female(X)`, write a structure that defines `mother(X, Y)`.

TR: *parent, male, female kullanarak mother(X, Y)'yi tanımla.*

```
mother(M, C) :- parent(M, C), female(M).
```

`mother(M, C)` — "M, C'nin annesidir"; yani M, C'nin parent'ıdır VE M kadındır.

2. Using the structures parent(X, Y), male(X), and female(X), write a structure that defines sister(X, Y).

TR: *parent, male, female ile sister(X, Y)'yi tanımla.*

```
sister(S, P) :- parent(M, S), parent(M, P), female(S), S \= P.
```

Açıklama:

- Ortak bir parent M var (S ve P'nin ikisi de M'in çocuğu).
- S kadın.
- $S \neq P$ (kendisi kendisinin sister'ı değil).

Daha sağlam versiyon — **iki parent'ın da ortak olmasını** isteyen:

```
sister(S, P) :- parent(F, S), parent(F, P),  
                parent(M, S), parent(M, P),  
                F \= M, female(S), S \= P.
```

Half-sister vs full-sister ayrımı için kullanılabilir. Genelde tek ortak parent yeterli.

3. Write a Prolog program that finds the maximum of a list of numbers.

TR: *Sayı listesinin maximum'unu bul.*

```
max_list([X], X).                                % tek elemanlı liste → o eleman  
max_list([H|T], Max) :-  
    max_list(T, MaxT),  
    (H >= MaxT -> Max = H ; Max = MaxT).
```

% Veya açıkça iki clause ile:

```
max_list([H|T], H) :-  
    max_list(T, MaxT),  
    H >= MaxT.  
max_list([H|T], MaxT) :-  
    max_list(T, MaxT),  
    H < MaxT.
```

Kullanım:


```
?- max_list([3, 1, 4, 1, 5, 9, 2, 6], M).  
M = 9.
```

Tail-recursive ve daha verimli versiyon:

```
max_list([H|T], Max) :- max_helper(T, H, Max).  
max_helper([], CurMax, CurMax).  
max_helper([H|T], CurMax, Max) :-  
    (H > CurMax -> max_helper(T, H, Max) ; max_helper(T, CurMax, Max)).
```

4. Write a Prolog program that succeeds if the intersection of two given list parameters is empty.

TR: İki listenin kesişimi boşsa başarılı olan program.

```
disjoint([], _). % boş liste herhangi şeyle disjoint  
disjoint([H|T], L) :-  
    \+ member(H, L), % H, L'de YOK  
    disjoint(T, L). % kalanlar da L'de yok  
  
% (member/2 zaten built-in; aksi hâlde:)  
member(X, [X|_]).  
member(X, [_|T]) :- member(X, T).
```

Kullanım:

```
?- disjoint([1, 2, 3], [4, 5, 6]).  
true.  
  
?- disjoint([1, 2, 3], [3, 4, 5]).  
false.
```

5. Write a Prolog program that returns a list containing the union of the elements of two given lists.

TR: İki listenin (set olarak) union'unu döndür.

```

union([], L, L).
union([H|T], L, R) :-
    member(H, L), !,                % H zaten L'de → R'ye ekleme
    union(T, L, R).
union([H|T], L, [H|R]) :-
    union(T, L, R).                % H L'de yok → R'ye ekle

```

Kullanım:

```

?- union([1, 2, 3], [3, 4, 5], R).
R = [1, 2, 3, 4, 5].

?- union([], [a, b], R).
R = [a, b].

```

Cut (!) ikinci clause'un başında: H zaten L'deyse üçüncü clause'a düşmesini engelle (aksi takdirde duplicate olur).

6. Write a Prolog program that returns the final element of a given list.

TR: *Listenin son elemanını döndür.*

```

last_elem([X], X).
last_elem([_|T], X) :- last_elem(T, X).

```

Kullanım:

```

?- last_elem([a, b, c, d], X).
X = d.

?- last_elem([42], X).
X = 42.

```

Algoritma: Liste tek elemanlıysa o eleman. Değilse tail'e recurse.

ISO Prolog'da built-in `last/2` zaten mevcut.

7. Write a Prolog program that implements quicksort.

TR: *Prolog'da quicksort yaz.*

```
quicksort([], []).
quicksort([Pivot|Rest], Sorted) :-
    partition(Pivot, Rest, Less, Greater),
    quicksort(Less, SortedLess),
    quicksort(Greater, SortedGreater),
    append(SortedLess, [Pivot|SortedGreater], Sorted).

partition(_, [], [], []).
partition(Pivot, [H|T], [H|Less], Greater) :-
    H <= Pivot, !,
    partition(Pivot, T, Less, Greater).
partition(Pivot, [H|T], Less, [H|Greater]) :-
    H > Pivot,
    partition(Pivot, T, Less, Greater).

% append built-in
% (manuel:)
append([], L, L).
append([H|T], L, [H|R]) :- append(T, L, R).
```

Kullanım:

```
?- quicksort([3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5], S).
S = [1, 1, 2, 3, 3, 4, 5, 5, 5, 6, 9].
```

Klasik quicksort:

- İlk eleman = pivot.
- Rest'i pivot'tan küçük/eşit (Less) ve büyük (Greater) parçalara böl.
- Her parçayı recursive sırala.
- Sıralanmış parçaları pivot ile birleştir.

Average $O(n \log n)$, worst $O(n^2)$ (sorted input + first-element pivot).

Sonuç

Bu doküman Sebesta'nın final kapsamında olan üç bölümün **tüm** end-of-chapter sorularını kapsar:

Bölüm	Review	Problem Set	Programming Ex	Toplam
Ch 8 — Control Structures	31	12	9	52
Ch 15 — Functional (Scheme)	50	10	22	82
Ch 16 — Logic (Prolog)	25	10	7	42
Genel toplam	106	32	38	176

Finalde tipik patterns:

- **Ch 8:** Goto'suz/break'siz yeniden yazım (PE 4 = Quiz 12A), Boolean vs arithmetic condition (Problem 9 = Quiz 12B), loop semantics fark anlatımı.
- **Ch 15:** Verilen Scheme fonksiyonunu trace etme / ne hesapladığını söyleme (Problem 9, 10 = Quiz 11A tarzı), tail recursive'e dönüştürme (PE 20 = Quiz 11A (c)), list işleme algoritması yazma.
- **Ch 16:** Prolog program trace (Quiz 11B tarzı), unification açıklama, `is` operatörü kısıtı (Review 17), aile ilişkileri rule'ları (Problem 3-5).

Çalışma önerisi:

1. Önce Quiz 11A, 11B, 12A, 12B çözümlerini iyice anla — finalde bu kalıplar doğrudan gelir.
2. Sonra her bölümün review sorularını gözden geçir — kavramsal sorular finalin "kısa cevap" kısmı.
3. Programming exercise'lardan **PE 5-12 (Ch 15)** ve **PE 3-7 (Ch 16)** önemli — bunlar gerçek liste işleme kalıpları.
4. Trace egzersizlerini (Problem 9-10 Ch 15) kâğıda kendin yaz — pasif okumayla anlaşılmaz.

Bol şans!